



Tecnun Universidad de Navarra

Proyecto Fin de Máster

ANÁLISIS DE DATOS EN INGENIERÍA

Comparación de codificadores visuales en Diffusion Policy: protocolo con prueba disjunta, rendimiento y coste en Push-T

Moises Britez

Director: Diego Borro

Donostia-San Sebastián, septiembre de 2026

Pº Manuel Lardizabal, 13. 20018 Donostia-San Sebastián, Gipuzkoa
Tel. 943 219 877 · Fax 943 311 442 · www.tecnun.es

Comparación de codificadores visuales en Diffusion Policy: protocolo con prueba disjunta, rendimiento y coste en Push-T

Memoria presentada por Moises Britez para la obtención del título de Máster Universitario en análisis de datos en ingeniería.

Director: Diego Borro

Fdo.: Moises Britez

Fdo.: el director

Donostia-San Sebastián, septiembre de 2026

AGRADECIMIENTOS

A Diego Borro, director de este trabajo, por la orientación recibida durante su desarrollo y por la exigencia con la que ha revisado cada versión del documento.

Al profesorado del Máster Universitario en Análisis de Datos en Ingeniería de Tecnun, Universidad de Navarra, por la formación que ha hecho posible abordar este estudio, y a mis compañeros de promoción por el trabajo compartido a lo largo de estos meses.

A la Administración Nacional de Electricidad (ANDE), por el apoyo institucional que ha permitido compatibilizar la actividad profesional con la realización del máster.

A mi familia y a mis amigos, por el respaldo constante y por la paciencia durante los meses de dedicación que ha requerido esta memoria.

ÍNDICE DE CONTENIDOS

Agradecimientos	iii
Resumen del proyecto	xi
Abstract	xiii
1. Introducción y objetivos	1
1.1 Contexto y motivación	1
1.2 Problema abordado	1
1.3 Objetivos	2
1.4 Alcance del trabajo	2
1.5 Estructura de la memoria	3
2. Estado del arte	5
2.1 Aprendizaje por imitación en manipulación robótica	5
2.2 Fundamentos de los modelos de difusión	5
2.3 Diffusion Policy para control visuomotor	6
2.4 Evolución de las políticas de difusión	7
2.5 Codificadores visuales y preentrenamiento	8
2.5.1 Redes residuales y supervisión en ImageNet	8
2.5.2 Transformadores visuales y aprendizaje autosupervisado	9
2.5.3 Preentrenamiento multimodal con CLIP	9
2.5.4 Representaciones visuales preentrenadas para control	9
2.5.5 Estrategias de adaptación	10
2.6 Síntesis y hueco de investigación	10
3. Metodología	13
3.1 Planteamiento experimental	13
3.2 Tarea y conjunto de datos	14
3.3 Arquitectura de la política	17
3.4 Variantes del codificador visual	18
3.5 Protocolo de entrenamiento	20
3.6 Protocolo de evaluación	22
3.7 Métricas	23
3.8 Selección del punto de control y prueba final	24
3.9 Entorno de ejecución y reproducibilidad	26
3.10 Coste temporal de los entrenamientos	29
4. Resultados	31
4.1 Cobertura de las ejecuciones	31

4.2	Evolución del rendimiento	32
4.3	Prueba final sobre semillas disjuntas	33
4.4	Ajuste y generalización	35
4.5	Coste computacional	36
4.6	Comprobación cualitativa	38
4.7	Contraste de las expectativas previas	39
4.8	Limitaciones	40
5.	Conclusiones	43
5.1	Conclusiones del trabajo	43
5.2	Trabajo futuro	46
6.	Presupuesto	49
6.1	Material fungible y suministros	49
6.2	Amortización de los equipos	49
6.3	Licencias de software	50
6.4	Mano de obra	50
6.5	Resumen del presupuesto	50
	Referencias	53

ÍNDICE DE FIGURAS

Figura 1.	Caracterización del conjunto de demostraciones de Push-T.	16
Figura 2.	Flujo de ajuste, validación, selección y prueba.	24
Figura 3.	Evolución de la puntuación media de evaluación de las cinco variantes.	33
Figura 4.	Evolución de las pérdidas de entrenamiento y validación.	36
Figura 5.	Inferencia de V1 y V2 sobre una condición inicial compartida.	39

ÍNDICE DE TABLAS

Tabla 1.	Resultado de la ablación de codificadores visuales de <i>Diffusion Policy</i> en la tarea <i>Square</i>	7
Tabla 2.	Principales líneas de extensión de <i>Diffusion Policy</i>	8
Tabla 3.	Reparto del conjunto de demostraciones de Push-T y características de cada subconjunto. La puntuación es la que alcanza el operador humano según la Ecuación (6).	15
Tabla 4.	Controles de integridad del archivo de demostraciones.	17
Tabla 5.	Configuración de las cinco variantes del codificador visual evaluadas.	19
Tabla 6.	Cadena de preprocesado de la imagen por variante, desde el fotograma renderizado hasta el tensor que recibe la red.	20
Tabla 7.	Hiperparámetros de entrenamiento de las cinco variantes.	21
Tabla 8.	Configuración efectiva de las cinco ejecuciones. Solo se muestran los parámetros que difieren entre ellas.	22
Tabla 9.	Plataforma de ejecución de los experimentos.	27
Tabla 10.	Componentes de la pila de software y función de cada uno.	27
Tabla 11.	Identificadores de los pesos de partida y huellas de los artefactos evaluados.	28
Tabla 12.	Coste temporal de los entrenamientos ejecutados.	29
Tabla 13.	Cobertura de los entrenamientos y mejor puntuación en el conjunto de selección. Los valores son estimaciones optimistas, no resultados finales.	31
Tabla 14.	Lecturas del conjunto de selección que no dependen del máximo.	32
Tabla 15.	Puntuación sobre el bloque de prueba disjunto, 200 condiciones por variante.	34
Tabla 16.	Diferencias pareadas sobre el bloque de prueba disjunto.	34
Tabla 17.	Latencia de inferencia de las cinco variantes, en milisegundos.	37
Tabla 18.	Pico de memoria gráfica reservada, en gibibytes.	38
Tabla 19.	Alcance de las conclusiones: qué queda demostrado y qué no.	46
Tabla 20.	Presupuesto de material fungible y suministros.	49
Tabla 21.	Amortización de los equipos imputada al proyecto.	50
Tabla 22.	Dedicación del autor por fases del proyecto.	51
Tabla 23.	Presupuesto de mano de obra.	51
Tabla 24.	Resumen del presupuesto.	51

RESUMEN DEL PROYECTO

Diffusion Policy genera acciones de manipulación robótica mediante un proceso de difusión condicionado por la observación. Su variante visuomotora encadena un codificador visual y una red generativa, de modo que la representación de la imagen delimita la información disponible para el control. La investigación posterior ha modificado sobre todo el bloque generativo, mientras que la elección del codificador y de su estrategia de entrenamiento sigue sin resolverse cuando solo se dispone de un conjunto reducido de demostraciones.

Este trabajo compara cinco bloques visuales integrados como codificador de observaciones de una misma *Diffusion Policy* en la tarea Push-T: una ResNet-18 entrenada desde cero, la misma red preentrenada sobre ImageNet en versiones congelada y con ajuste fino, y dos transformadores de visión congelados, DINOv2 ViT-S/14 y CLIP ViT-B/16. Todos entregan un vector de 512 componentes y comparten red generativa, datos y protocolo de evaluación. El ajuste emplea 90 de las 206 demostraciones teleoperadas disponibles, según el criterio de la implementación de referencia, y presupuestos de 200 a 500 épocas, con parada anticipada. Un conjunto de 50 episodios generados con semillas reservadas selecciona el punto de control de cada variante. El rendimiento se mide después una sola vez sobre 200 episodios de un bloque de semillas disjunto, bajo un protocolo preregistrado, junto con el tiempo de cómputo consumido. Cada bloque visual se entrena una única vez, premisa de diseño adoptada para cubrir cinco configuraciones con el presupuesto disponible; en consecuencia, la unidad sobre la que se infiere es el artefacto entrenado y no la estrategia de entrenamiento.

Sobre ese bloque independiente, la línea base entrenada desde cero alcanza 0,872. Las cuatro variantes preentrenadas obtienen 0,649, 0,586, 0,578 y 0,490. V0 supera a todas con significación tras corregir los diez contrastes por pares mediante el procedimiento de Holm, y ese resultado no depende de qué prueba se emplee. Entre las variantes preentrenadas, en cambio, solo la peor se distingue de las demás, y el orden entre las tres restantes queda sin resolver. Cada época de V0 ocupa 1,6 min, frente a un intervalo de 2,3 min a 14,7 min en las demás. Las cinco ejecuciones suman 237,1 h.

Se concluye que, entre los cinco artefactos evaluados y sobre las condiciones ensayadas, ninguno de los bloques preentrenados mejora el rendimiento ni el coste por época de la línea base. La ventaja debe atribuirse al bloque visual completo (pesos iniciales, resolución, recorte y agregación espacial), que varía de forma conjunta, y no al preentrenamiento aislado. Con una única ejecución de entrenamiento por variante, el resultado no autoriza a extrapolar el comportamiento esperado de esas estrategias al volver a entrenarlas.

Palabras clave: aprendizaje por imitación, modelos de difusión, codificador visual, transferencia de aprendizaje, manipulación robótica.

ABSTRACT

Diffusion Policy generates robotic manipulation actions through a diffusion process conditioned on the observation. Its visuomotor variant chains a visual encoder and a generative network, so the image representation bounds the information available for control. Subsequent research has mostly modified the generative block, whereas the choice of encoder and of its training strategy remains unsettled when only a small set of demonstrations is available.

This work compares five visual blocks integrated as the observation encoder of a single *Diffusion Policy* on the Push-T task: a ResNet-18 trained from scratch, the same network pretrained on ImageNet in frozen and fine-tuned versions, and two frozen vision transformers, DINOv2 ViT-S/14 and CLIP ViT-B/16. All of them output a 512-component vector and share the generative network, the data, and the evaluation protocol. Training uses 90 of the 206 teleoperated demonstrations, following the reference implementation, and budgets ranging from 200 to 500 epochs, with early stopping. A set of 50 episodes generated with held-out seeds selects the checkpoint of each variant. Performance is then measured once on 200 episodes from a disjoint seed block, under a preregistered protocol, together with the compute time consumed. Each visual block is trained exactly once, a design premise adopted in order to cover five configurations within the available compute budget; the unit of inference is therefore the trained artefact, not the training strategy.

On that independent block, the baseline trained from scratch reaches 0.872. The four pretrained variants score 0.649, 0.586, 0.578, and 0.490. V0 outperforms all of them after the ten pairwise tests are corrected with Holm’s procedure, and this holds regardless of which test is used. Among the pretrained variants, by contrast, only the weakest is distinguishable from the others, and the ordering of the remaining three is left unresolved. Each V0 epoch takes 1.6 min, compared with 2.3 min to 14.7 min for the other variants. The five runs total 237.1 h.

The study concludes that, among the five artefacts evaluated and over the conditions tested, none of the pretrained visual blocks improves the baseline either in performance or in cost per epoch. The advantage must be ascribed to the complete visual block (initial weights, resolution, cropping, and spatial aggregation), which varies jointly, rather than to pretraining in isolation. With a single training run per variant, the result does not license extrapolation to the expected behaviour of those strategies upon retraining.

Keywords: imitation learning, diffusion models, visual encoder, transfer learning, robotic manipulation.

1. INTRODUCCIÓN Y OBJETIVOS

Este capítulo sitúa el trabajo en su ámbito, delimita el problema que aborda y enuncia los objetivos que guían la investigación. Además, precisa el alcance del estudio y describe la organización de la memoria.

1.1 Contexto y motivación

La manipulación robótica requiere generar acciones continuas a partir de observaciones sensoriales de alta dimensión. El aprendizaje por imitación aborda esa dificultad ajustando una política sobre demostraciones humanas, sin necesidad de diseñar una función de recompensa [1].

Sin embargo, las demostraciones humanas son multimodales. Ante una misma observación, un operario puede ejecutar acciones distintas y todas ellas válidas. Los métodos de clonación de comportamiento basados en regresión promedian esas alternativas y producen trayectorias incoherentes [2].

Para evitar ese promediado, Chi *et al.* [3] proponen *Diffusion Policy*, que representa la acción como un proceso de difusión condicionado por la observación. El método hereda así la capacidad de los modelos generativos de difusión para representar distribuciones multimodales [4].

En su variante visuomotora, *Diffusion Policy* se compone de dos bloques. Un codificador visual transforma las imágenes en un vector de características, y una red generativa produce la secuencia de acciones condicionada por dicho vector. Por tanto, la representación visual determina qué información alcanza al bloque generativo.

1.2 Problema abordado

La investigación sobre *Diffusion Policy* ha modificado principalmente el bloque generativo y la modalidad de entrada. Algunos trabajos sustituyen las imágenes por nubes de puntos [5], mientras que otros reducen el coste de inferencia [6].

El trabajo original incluye una ablación de ResNet y CLIP en la tarea *Square* [3]. Además, un estudio posterior compara ResNet-18 y DINOv3 en cuatro tareas, incluida Push-T [7]. Sin embargo, ninguno contrasta conjuntamente ResNet-18, DINOv2 y CLIP bajo el mismo protocolo de Push-T.

De ahí la pregunta que guía el trabajo: dentro de una misma *Diffusion Policy* entrenada sobre un conjunto reducido de demostraciones de Push-T, ¿qué diferencia de rendimiento y de coste separa a cinco bloques visuales concretos, y con qué grado de certeza puede afirmarse esa diferencia?

Conviene precisar desde el principio en qué nivel se formula esa pregunta. Cada bloque visual se lleva a un único artefacto entrenado, es decir, a un punto de control concreto obtenido con una sola ejecución de entrenamiento. La comparación que este trabajo puede sostener es,

por tanto, la comparación de esos cinco artefactos sobre un conjunto de condiciones iniciales, y no la estimación del rendimiento esperado al repetir cada estrategia de entrenamiento. El Apartado 1.4 justifica esa restricción y el Apartado 3.8 la traslada a la especificación estadística.

1.3 Objetivos

El objetivo general consiste en cuantificar y comparar el rendimiento en bucle cerrado y el coste computacional de cinco bloques visuales integrados como codificador de observaciones de una misma *Diffusion Policy* en Push-T, cada uno entrenado una sola vez bajo un protocolo documentado, y en delimitar el alcance inferencial de esa comparación.

Este propósito se concreta en los objetivos específicos siguientes:

1. Implementar e integrar los cinco bloques visuales dentro de la misma *Diffusion Policy*, manteniendo constantes la red generativa, el conjunto de datos y la interfaz de condicionamiento.
2. Estimar, sobre un bloque de condiciones iniciales disjunto y preregistrado, las diferencias de rendimiento entre los cinco puntos de control, acompañadas de su incertidumbre y con control de la multiplicidad.
3. Caracterizar el coste computacional de cada variante en cuatro ejes: parámetros entrenables, tiempo por época, latencia de inferencia desglosada y pico de memoria gráfica.
4. Acotar la validez de la comparación: cuantificar el sesgo del conjunto de selección y establecer qué no puede concluirse a partir de una única ejecución de entrenamiento por variante.

1.4 Alcance del trabajo

El estudio se circunscribe a la tarea Push-T en simulación, que consiste en desplazar una pieza con forma de T hasta una posición objetivo mediante un efector circular [3]. Todas las variantes comparten la misma política generativa, el conjunto de datos y el protocolo de evaluación. La variable manipulada es el bloque visual completo, es decir, la arquitectura del codificador, sus pesos iniciales, su estrategia de adaptación y su cadena de preprocesado, que varían de forma conjunta. Las desviaciones de presupuesto y tamaño de lote se documentan de forma explícita.

La premisa de diseño que más condiciona el alcance es deliberada y se declara aquí: **cada variante se entrena una sola vez**, con la semilla 42. No se trata de una omisión, sino de una decisión de reparto del presupuesto de cómputo. Las cinco ejecuciones suman 237,1 h sobre la única GPU disponible, según el Apartado 3.10; replicar la matriz con tres semillas habría exigido unas 700 h, incompatibles con el calendario del trabajo. En consecuencia, la unidad sobre la que este estudio infiere es el *artefacto entrenado*, no la estrategia de entrenamiento: los intervalos y contrastes del Capítulo 4 describen cómo se comportan cinco puntos de control fijos ante nuevas condiciones iniciales, y no cómo se comportaría cada estrategia si volviera a entrenarse. El Apartado 3.8 formaliza este estimando y el Apartado 4.8 recoge sus consecuencias.

Quedan fuera del alcance cuatro aspectos. En primer lugar, la validación sobre un robot

físico, puesto que el trabajo opera únicamente en simulación. En segundo lugar, el análisis de robustez frente a perturbaciones visuales. En tercer lugar, la estimación de la variabilidad entre ejecuciones de entrenamiento, que exigiría varias semillas por variante. En cuarto lugar, las ablaciones factoriales que separarían la inicialización de los pesos del preprocesado y de la agregación espacial. Los dos últimos se concretan como líneas de continuación en el Apartado 5.2.

1.5 Estructura de la memoria

La memoria se organiza en seis capítulos. El Capítulo 2 revisa los fundamentos de los modelos de difusión, su aplicación al control visuomotor y las familias de codificadores visuales consideradas. El Capítulo 3 describe el diseño experimental, el conjunto de datos, las variantes evaluadas y las métricas.

A continuación, el Capítulo 4 presenta las curvas de entrenamiento, la comparativa de rendimiento y el coste computacional de cada variante. El Capítulo 5 interpreta esos resultados, señala las limitaciones del estudio y propone líneas de continuación. Por último, el Capítulo 6 detalla los costes del proyecto.

2. ESTADO DEL ARTE

Este capítulo revisa los fundamentos que permiten estudiar la influencia del codificador visual en una política de difusión. La exposición parte del aprendizaje por imitación y continúa con los modelos generativos de difusión. Después, se describe *Diffusion Policy* y se analizan sus principales extensiones.

A continuación, la revisión se centra en las representaciones visuales empleadas en este trabajo. Se consideran las redes residuales, los transformadores visuales y el preentrenamiento supervisado, autosupervisado y multimodal. El capítulo concluye delimitando el hueco experimental que aborda la metodología.

2.1 Aprendizaje por imitación en manipulación robótica

El aprendizaje por imitación obtiene una política a partir de demostraciones de un experto. Su formulación más directa, denominada clonación del comportamiento, trata cada par observación-acción como un ejemplo supervisado. Este planteamiento evita diseñar una función de recompensa, pero depende de la cobertura y calidad de las demostraciones [1], [8].

Sin embargo, el entrenamiento supervisado y la ejecución de la política siguen distribuciones diferentes. Un error modifica el estado visitado y puede llevar al robot fuera de los datos de entrenamiento. Los errores posteriores se acumulan porque la política carece de ejemplos en esas nuevas regiones. DAgger reduce este desplazamiento mediante la agregación iterativa de correcciones del experto [1].

Además, las demostraciones de manipulación suelen contener varias acciones válidas para una misma observación. Una pérdida cuadrática aplicada a una política determinista aproxima la media condicional de esas alternativas. Dicha media puede situarse entre modos válidos y generar una acción que no aparece en ninguna demostración [2].

Para representar esa multimodalidad, la clonación de comportamiento implícita define una función de energía sobre pares de observación y acción. La acción se obtiene minimizando esa energía durante la inferencia. Este enfoque mejora la expresividad, aunque requiere muestras negativas y una optimización adicional [2]. Estas limitaciones motivan el uso de modelos generativos directamente sobre las secuencias de control.

2.2 Fundamentos de los modelos de difusión

Los modelos de difusión aprenden a invertir una degradación gradual de los datos. Los modelos probabilísticos de difusión por eliminación de ruido (DDPM) definen un proceso directo que añade ruido gaussiano durante K pasos. Para una muestra limpia $\mathbf{x}^0$, un estado perturbado puede expresarse como [4]

$$q(\mathbf{x}^k | \mathbf{x}^0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_k} \mathbf{x}^0, (1 - \bar{\alpha}_k)\mathbf{I}\right). \quad (1)$$

En cambio, el proceso inverso parte de ruido y estima sucesivamente una muestra de la distribución original. Ho *et al.* [4] parametrizan una red que predice el ruido incorporado en cada paso. Su objetivo simplificado es un error cuadrático entre el ruido real y el estimado, lo que permite un entrenamiento estable.

Asimismo, los modelos basados en puntuación aprenden la dirección de mayor crecimiento de la densidad logarítmica. Este campo permite generar muestras mediante dinámica de Langevin [9]. La introducción de varios niveles de ruido facilita la estimación en regiones con pocos datos.

Por otra parte, las ecuaciones diferenciales estocásticas (SDE) ofrecen una formulación continua común para estos procesos. El proceso inverso depende de la función de puntuación de las distribuciones perturbadas. Esta perspectiva conecta los DDPM con otros modelos basados en puntuación y amplía las alternativas de muestreo [10].

No obstante, el muestreo iterativo aumenta la latencia. Los modelos implícitos de difusión por eliminación de ruido (DDIM) conservan el objetivo de entrenamiento de los DDPM, pero emplean un proceso inverso no markoviano. Esta modificación reduce el número de evaluaciones necesarias durante la generación [11].

En conjunto, estos trabajos proporcionan tres elementos para el control robótico: una distribución generativa expresiva, un objetivo de predicción de ruido y un procedimiento iterativo de muestreo. *Diffusion Policy* traslada esos elementos desde la generación de datos al espacio de acciones.

2.3 Diffusion Policy para control visuomotor

Diffusion Policy modela la distribución condicional $p(\mathbf{A}_t \mid \mathbf{O}_t)$, donde $\mathbf{O}_t$ contiene las observaciones recientes y $\mathbf{A}_t$ representa una secuencia de acciones futuras [3]. Durante el entrenamiento, se perturba una secuencia demostrada y se minimiza la pérdida

$$\mathcal{L}_{DP} = \mathbb{E}_{k, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{O}_t, \mathbf{A}_t^k, k)\|_2^2 \right]. \quad (2)$$

De este modo, la red ϵ_{θ} aprende a eliminar el ruido de una trayectoria condicionada por la observación. La formulación admite varios modos de acción sin promediarlos en una única salida. Además, evita estimar la constante de normalización que dificulta el entrenamiento de los modelos de energía [3].

En cuanto a la arquitectura, el trabajo original propone dos redes para predecir el ruido. La primera es una red convolucional temporal con estructura U-Net y modulación lineal por características (FiLM). La segunda emplea un transformador temporal con atención cruzada. La variante convolucional ofrece mayor estabilidad, mientras que el transformador responde mejor ante cambios de acción rápidos [3].

Además, la política aplica control de horizonte recesivo. A partir de una ventana de observaciones, predice una secuencia completa y ejecuta solo sus primeras acciones. Después, incorpora una nueva observación y repite la predicción. Esta estrategia combina coherencia temporal con capacidad de reacción ante desviaciones [3].

En la rama visual, cada imagen se transforma en un vector antes del proceso de eliminación de ruido. La implementación de referencia utiliza una ResNet-18 sin preentrenamiento. El

promedio espacial global se sustituye por un *spatial softmax* para conservar información posicional. Asimismo, la normalización por lotes se reemplaza por normalización por grupos para estabilizar el entrenamiento [3].

La influencia de esta rama ya se examina parcialmente en el artículo original. La Tabla 1 resume la comparación realizada sobre la tarea *Square* de RoboMimic. Se mantienen fijos el conjunto de demostraciones y la política convolucional, pero cambian la arquitectura y la estrategia de adaptación del codificador.

Tabla 1. Resultado de la ablación de codificadores visuales de *Diffusion Policy* en la tarea *Square*.

Codificador y preentrenamiento	Desde cero	Congelado	Ajuste fino
ResNet-18, ImageNet-21k	0,94	0,58	0,92
ResNet-34, ImageNet-21k	0,92	0,40	0,94
ViT-B/16, CLIP	0,22	0,70	0,98

Fuente: adaptado de [3, Tabla 5].

Como muestra la tabla, ninguna estrategia domina para todas las arquitecturas. El ajuste fino obtiene el mejor resultado con ViT-B/16, mientras que ResNet-18 desde cero supera ligeramente a su versión ajustada. En cambio, la congelación reduce el rendimiento de ambas redes residuales [3].

Por último, Push-T permite estudiar estas decisiones en una tarea de contacto planar. Un efector circular debe empujar una pieza con forma de T hasta una región objetivo. La puntuación depende del solapamiento entre la pieza y dicha región, por lo que exige controlar posición y orientación [2], [3]. En la versión física, la política entrenada de extremo a extremo alcanzó un 0,95 de éxito, frente a 0,80 con R3M congelado y 0,15 con ImageNet congelado [3]. Estos resultados confirman que la representación visual puede alterar el control aun cuando la red generativa permanece fija.

2.4 Evolución de las políticas de difusión

Tras la publicación de *Diffusion Policy*, la investigación se ha dividido en varias líneas. Unos trabajos modifican la representación sensorial, mientras que otros actúan sobre la jerarquía, la geometría o la velocidad de inferencia. La Tabla 2 sintetiza las extensiones más relacionadas con este estudio.

En primer lugar, DP3, HDP y ET-SEED incorporan estructura geométrica mediante nubes de puntos o restricciones cinemáticas [5], [12], [14]. Estas propuestas mejoran la generalización espacial, pero cambian simultáneamente la modalidad y la arquitectura perceptiva. Por ello, sus resultados no permiten aislar el efecto de un codificador de imagen 2D.

En segundo lugar, *Consistency Policy* reduce el coste del muestreo mediante destilación. El trabajo mantiene constante la codificación de imagen y concentra la comparación en la etapa generativa [6]. De forma análoga, la percepción activa modifica el espacio de control para coordinar la cámara y el manipulador [13].

Finalmente, el estudio DINOv3-DP es el antecedente más próximo a esta investigación. Compara ResNet-18 y un ViT-S/16 autosupervisado en Push-T, Lift, Can y Square. Además, evalúa entrenamiento desde cero, congelación y ajuste fino durante 100 épocas [7]. Sus

Tabla 2. Principales líneas de extensión de *Diffusion Policy*.

Trabajo	Componente modificado	Representación	Contribución principal
DP3 [5]	Percepción	Nube de puntos	Sustituye la imagen 2D por una representación geométrica 3D compacta.
HDP [12]	Jerarquía y cinemática	RGB-D y nube de puntos	Separa la planificación de alto nivel del control difusivo local.
Consistency Policy [6]	Inferencia	Imagen 2D	Destila la trayectoria de eliminación de ruido para reducir la latencia.
Percepción activa [13]	Espacio de estado y acción	Cámara móvil RGB	Coordina el punto de vista mediante una restricción cinemática.
ET-SEED [14]	Equivarianza geométrica	Nube de puntos	Introduce simetría SE(3) en la generación de trayectorias.
DINOv3-DP [7]	Codificador visual	Imagen 2D	Compara ResNet-18 y DINOv3 bajo tres estrategias de entrenamiento.

Fuente: elaboración propia.

resultados muestran que la ventaja del preentrenamiento depende de la tarea y de la estrategia de adaptación.

2.5 Codificadores visuales y preentrenamiento

La revisión anterior muestra que la imagen condiciona toda la secuencia de acciones. Por tanto, el codificador debe conservar la geometría relevante y descartar variaciones que no afecten a la tarea. Las familias seleccionadas difieren tanto en su arquitectura como en la señal utilizada durante el preentrenamiento.

2.5.1 Redes residuales y supervisión en ImageNet

Las redes residuales aprenden funciones de corrección respecto a la entrada de cada bloque. Las conexiones de identidad facilitan la optimización de redes profundas y permiten reutilizar características en distintas escalas [15]. En particular, ResNet-18 ofrece un compromiso entre capacidad y coste que favorece su uso en políticas visuomotoras.

Asimismo, el preentrenamiento supervisado en ImageNet emplea etiquetas de objeto sobre un conjunto visual amplio [16]. Estos pesos proporcionan detectores generales de textura, forma y categoría. Sin embargo, la clasificación no exige conservar toda la posición espacial que requiere una tarea de contacto.

Por esta razón, una representación adecuada para reconocimiento puede no serlo para control.

2.5.2 Transformadores visuales y aprendizaje autosupervisado

En cambio, el transformador visual (ViT) divide la imagen en parches y los procesa como una secuencia de *tokens*. La autoatención relaciona regiones alejadas sin depender exclusivamente de la localidad convolucional. Su rendimiento mejora cuando existe un preentrenamiento a gran escala [17].

Sobre esta arquitectura, DINOv2 aprende mediante autodestilación sin etiquetas. El método combina datos visuales diversos y curados con una estrategia de entrenamiento autosupervisado. Sus características transfieren a tareas de imagen y de nivel de píxel sin ajuste específico [18].

En consecuencia, DINOv2 resulta pertinente para manipulación con pocas demostraciones. Sus representaciones densas pueden aportar correspondencias espaciales aprendidas fuera del dominio robótico. Sin embargo, la congelación impide adaptar esas características a la dinámica y a los contactos de Push-T. La comparación con DINOv3 confirma que esta decisión debe evaluarse para cada tarea [7].

2.5.3 Preentrenamiento multimodal con CLIP

Por su parte, el preentrenamiento contrastivo de imagen y lenguaje (CLIP) alinea una imagen con su descripción textual. El modelo original utiliza 400 millones de pares obtenidos de Internet y transfiere sus representaciones a distintas tareas visuales [19]. La supervisión lingüística favorece características con contenido semántico general.

Sin embargo, Push-T no requiere interpretar instrucciones ni reconocer categorías abiertas. La política necesita localizar una pieza, estimar su orientación y relacionarla con una meta. Por tanto, la semántica de CLIP solo resultará útil si su representación conserva también la información geométrica necesaria.

Además, la ablación original muestra una diferencia marcada entre congelación y ajuste fino. El ViT-B/16 preentrenado con CLIP obtiene 0,70 cuando permanece fijo y 0,98 cuando se adapta [3]. Este resultado impide suponer que un corpus de preentrenamiento mayor garantiza por sí solo una política mejor.

2.5.4 Representaciones visuales preentrenadas para control

Frente al preentrenamiento visual genérico, otra línea utiliza datos y objetivos relacionados con la interacción. R3M aprende de vídeo egocéntrico mediante objetivos temporales y alineación con lenguaje. Como módulo congelado, mejora varias tareas de manipulación con pocos datos [20]. No obstante, en Push-T queda por debajo del codificador entrenado de extremo a extremo [3].

Asimismo, MVP preentrena un ViT mediante modelado enmascarado sobre imágenes de interacción humana. Después, mantiene fijo el codificador y aprende controladores mediante aprendizaje por refuerzo. En las ocho tareas de PixMC, esta estrategia supera al preentrenamiento supervisado en ImageNet en siete casos [21]. Sin embargo, el estudio no utiliza demostraciones ni una política de difusión.

Por su parte, Voltron combina reconstrucción visual enmascarada con supervisión lingüística. Su evaluación sobre cinco problemas robóticos muestra que las representaciones reconstructivas favorecen detalles espaciales, mientras que los objetivos contrastivos capturan

mejor la semántica. El equilibrio entre ambos niveles depende de la tarea, y una mayor carga semántica puede perjudicar algunos problemas de control de bajo nivel [22].

Finalmente, CortexBench compara CLIP, R3M, MVP y VIP sobre 17 tareas de siete bancos de pruebas. Ninguna representación obtiene el mejor resultado en todos los dominios. VC-1 alcanza el mejor promedio, pero tampoco domina cada tarea; además, su adaptación específica produce mejoras sustanciales [23]. Por tanto, la escala del preentrenamiento no determina por sí sola la utilidad de un codificador para control.

2.5.5 Estrategias de adaptación

En términos generales, el entrenamiento desde cero adapta toda la representación a la tarea, pero depende exclusivamente de las demostraciones disponibles. Esta opción reduce el sesgo entre preentrenamiento y control a costa de una mayor necesidad de datos.

En cambio, un codificador congelado conserva el conocimiento previo y reduce los parámetros entrenables. También evita que un conjunto pequeño degrade las características iniciales. Su principal limitación es que la política solo puede adaptar las capas posteriores a una representación fija.

Por último, el ajuste fino permite adaptar los pesos preentrenados. Esta estrategia ofrece mayor flexibilidad, aunque incrementa la memoria necesaria y puede producir sobreajuste. La evidencia disponible no establece una regla universal: el resultado cambia con la arquitectura, la tarea y el dominio de preentrenamiento [3], [7].

2.6 Síntesis y hueco de investigación

En conjunto, la literatura confirma que las políticas de difusión representan acciones multimodales y mantienen coherencia temporal. Los trabajos posteriores han mejorado la geometría, la jerarquía y la velocidad de inferencia. Sin embargo, la representación visual 2D suele permanecer fija o cambia junto con otros componentes.

De forma complementaria, los estudios sobre representaciones preentrenadas para control descartan la existencia de un codificador universalmente superior. El resultado depende del dominio, del objetivo de preentrenamiento y de la estrategia de adaptación [21], [22], [23].

El artículo original constituye una excepción, puesto que compara ResNet-18, ResNet-34 y CLIP ViT-B/16 bajo tres estrategias. No obstante, esa ablación se limita a la tarea *Square* [3]. En Push-T físico solo se contrastan el entrenamiento de extremo a extremo y dos codificadores congelados, ImageNet y R3M.

Asimismo, DINOv3-DP amplía la evidencia a cuatro tareas e incluye Push-T. Ese estudio no evalúa DINOv2 ni incorpora CLIP en la misma matriz comparativa [7]. Además, utiliza 100 épocas y una configuración de red distinta de la implementación original empleada en este trabajo.

Los estudios generales de representaciones tampoco cierran este hueco, puesto que emplean aprendizaje por refuerzo o clonación de comportamiento sobre bancos de pruebas heterogéneos. Ninguno mantiene fija una *Diffusion Policy* para comparar estas alternativas bajo un protocolo común de Push-T.

Por tanto, en el corpus revisado no se ha identificado una comparación conjunta de cinco configuraciones concretas: ResNet-18 desde cero, ResNet-18 preentrenada en ImageNet congelada y ajustada, DINOv2 congelado y CLIP congelado. Tampoco se informa su relación entre rendimiento, parámetros entrenables, tiempo y memoria bajo un protocolo único de Push-T.

Este hueco justifica un estudio controlado que mantenga constante la política generativa, el conjunto de demostraciones y la evaluación. La variable experimental queda así restringida al bloque visual completo, entendido como la arquitectura del codificador junto con sus pesos iniciales, su estrategia de adaptación y su cadena de preprocesado, que en la práctica varían de forma conjunta. El Capítulo 3 presenta el diseño empleado para realizar esa comparación y delimita hasta dónde alcanza.

3. METODOLOGÍA

Este capítulo describe el diseño experimental empleado para responder a los objetivos del Capítulo 1. En primer lugar, expone el planteamiento comparativo y las variables implicadas. A continuación, detalla la tarea, el conjunto de datos y la arquitectura de la política. Por último, define las cinco variantes evaluadas, los protocolos de entrenamiento y evaluación, las métricas, el entorno de ejecución y el coste temporal de los experimentos.

3.1 Planteamiento experimental

El estudio adopta un diseño comparativo centrado en una variable de interés. Esa variable es el bloque visual de la política, entendido como el codificador, sus pesos y su cadena de preprocesado. La arquitectura generativa, los datos y la evaluación permanecen constantes; el presupuesto y el tamaño de lote presentan las desviaciones descritas en el Apartado 3.5.

El planteamiento parte de tres **expectativas previas**, no de hipótesis estadísticas. La distinción es deliberada: una hipótesis sobre estrategias de entrenamiento exigiría varias ejecuciones por estrategia, y este diseño dispone de una sola. Las expectativas se enuncian, por tanto, sobre las configuraciones completas que se comparan, y su papel es orientar la lectura de los resultados, no ser sometidas a contraste formal.

- E1. Las configuraciones cuyo codificador parte de pesos preentrenados sobre un corpus amplio de imágenes superarán a la configuración entrenada desde cero, puesto que el número de demostraciones disponibles es reducido.
- E2. Entre las dos configuraciones que comparten arquitectura ResNet-18 preentrenada, la que actualiza los pesos del codificador superará a la que los mantiene congelados, al adaptar la representación a la distribución visual sintética de la tarea.
- E3. Las configuraciones basadas en transformador de visión (*Vision Transformer*, ViT) presentarán un coste de inferencia del codificador superior al de las convolucionales. El orden interno entre las cuatro configuraciones preentrenadas se deja abierto: el diseño no incluye margen de equivalencia ni prueba dirigida a demostrarlo, de modo que ese orden puede quedar sin resolver.

El Apartado 4.7 recupera las tres expectativas y las confronta con la evidencia obtenida, indicando en cada caso el nivel al que la afirmación resulta sostenible.

Para que la comparación resulte interpretable, se mantienen constantes cinco factores: la red generativa y sus hiperparámetros, el conjunto de datos y su partición, la semilla de entrenamiento, el protocolo de evaluación y la dimensión del vector de condicionamiento a la salida del codificador. Esta última restricción resulta esencial: todas las variantes entregan un vector de 512 componentes, de modo que la red generativa recibe siempre una entrada de la misma dimensión.

A esos factores constantes se suma una **premisa de diseño** que conviene enunciar antes que ningún resultado: cada variante se entrena una sola vez, con la semilla 42. La premisa responde al reparto del presupuesto de cómputo. Una ejecución ocupa entre 11 y 96 horas

de la única unidad de procesamiento gráfico (*Graphics Processing Unit*, GPU) disponible y las cinco suman 237,1 h, según el Apartado 3.10; con tres semillas por variante el cómputo rondaría las 700 h, incompatibles con el calendario del trabajo. Ante la alternativa entre cubrir cinco bloques visuales con una semilla o dos bloques con tres semillas, se optó por la cobertura.

La consecuencia es directa y determina toda la especificación estadística del Apartado 3.8: la unidad experimental sobre la que se infiere es el artefacto entrenado, es decir, el punto de control resultante, y no la estrategia de entrenamiento que lo produjo. Los contrastes de este trabajo comparan cinco puntos de control fijos ante nuevas condiciones iniciales; no cuantifican la variabilidad que introduciría volver a entrenar la misma configuración con otra semilla. Toda lectura de los resultados debe respetar ese nivel.

3.2 Tarea y conjunto de datos

La tarea de evaluación es Push-T, propuesta junto con *Diffusion Policy* [3]. Un efector circular debe empujar una pieza rígida con forma de T sobre un plano hasta hacerla coincidir con una silueta objetivo. La tarea exige contacto intermitente y admite varias secuencias de empuje válidas, de ahí su carácter multimodal.

La política observa dos modalidades: una imagen en color de 96×96 píxeles renderizada desde una vista cenital y la posición bidimensional del efector. La acción es la posición objetivo del efector, es decir, se emplea control en posición y no en velocidad. El entorno resuelve la dinámica de contacto mediante un motor de cuerpos rígidos bidimensional.

El conjunto de datos es el publicado por los autores del método, en formato `zarr` y con un tamaño aproximado de 30 megabytes. Contiene 206 demostraciones teleoperadas por un operador humano, que suman 25.650 transiciones. Cada transición almacena la imagen, el estado del efector, los puntos característicos de la pieza, la acción ejecutada y el número de contactos.

Cada demostración arranca en la condición inicial que el simulador genera a partir de una semilla propia, y esas semillas recorren los valores de 0 a 205. El dato se ha verificado reproduciendo con el entorno el primer instante de las 206 demostraciones, que se recupera sin error apreciable. Las semillas reservadas para la evaluación, descritas en el Apartado 3.6, quedan por tanto fuera de ese rango.

La partición de los datos reproduce la de la implementación de referencia y se resume en la Tabla 3. Un sorteo aleatorio reserva el dos por ciento de los episodios para validación, cuatro en total, y un segundo sorteo escoge 90 de los 202 restantes para el entrenamiento. Los 112 episodios no seleccionados, el 54,4 % del conjunto, no intervienen en el ajuste. La semilla de ambos sorteos está fijada en 42 dentro de la configuración de la tarea y es independiente de la semilla de entrenamiento, de modo que las cinco variantes emplean exactamente los mismos episodios.

El tope de 90 episodios procede de la implementación de referencia y define el régimen de pocas demostraciones que motiva las expectativas del Apartado 3.1. Ese régimen admite una descripción cuantitativa. Los episodios de entrenamiento aportan 11.356 transiciones, equivalentes a 18,9 min de teleoperación de los 42,8 min que contiene el fichero. Con el horizonte de 16 instantes, el muestreador extrae de ellos 10.726 ventanas, que a lote efectivo 64 producen 168 actualizaciones por época.

Tabla 3. Reparto del conjunto de demostraciones de Push-T y características de cada subconjunto. La puntuación es la que alcanza el operador humano según la Ecuación (6).

Subconjunto	Episodios	Transiciones	Ventanas	Longitud media	Puntuación media
Entrenamiento	90	11.356	10.726	126,2	0,894
Validación	4	432	404	108,0	0,893
Descartados	112	13.862	—	123,8	0,891
Total	206	25.650	—	124,5	0,892

Fuente: elaboración propia.

La calidad de las demostraciones acota lo que la política puede aprender. Al aplicar a cada una la métrica definida en el Apartado 3.7, el operador humano obtiene una puntuación media de 0,892, con mediana 0,896 y recorrido entre 0,813 y 0,949. Ninguna de las 206 demostraciones alcanza el umbral de cobertura que el entorno considera un éxito, de manera que el valor unitario de la métrica no está representado en los datos. Esa cifra es una referencia y no una cota superior estricta, puesto que las condiciones iniciales de las demostraciones no coinciden con las de la evaluación.

La Figura 1 reúne las cuatro distribuciones que describen el conjunto: longitud de los episodios, puntuación del demostrador, posición inicial de la pieza y orientación inicial de la pieza.

El argumento que descarta un sesgo de selección es el procedimiento, no una prueba estadística: el reparto responde a un sorteo aleatorio uniforme sobre los índices de episodio, sin ningún criterio de calidad, longitud ni dificultad. Un sorteo uniforme no puede introducir sesgo sistemático por construcción.

Las comparaciones entre los dos subconjuntos cumplen aquí un papel distinto y más modesto, el de comprobación descriptiva de que el sorteo se ejecutó como se describe. Se presentan como tamaños de efecto con su incertidumbre, y no como valores p , porque no rechazar la igualdad no demuestra equivalencia. Los episodios de entrenamiento son, en media, 2,4 transiciones más largos que los descartados, con un intervalo BCa al 95 % de $[-7,7; 12,6]$ sobre una longitud media de 126, es decir menos del 10 % de esa media en cualquiera de los dos extremos. En puntuación del demostrador la diferencia es de 0,004, con intervalo $[-0,004; 0,011]$, un orden de magnitud por debajo de las diferencias entre variantes que este trabajo interpreta. El δ de Cliff, que mide la probabilidad de que un episodio de un subconjunto supere a uno del otro, vale 0,026 en longitud y 0,085 en puntuación, ambos dentro de lo que se considera un efecto despreciable. El mismo criterio se aplica a la comparación entre las condiciones iniciales de entrenamiento y las 50 de evaluación, superpuestas en la Figura 1: los estadísticos de Kolmogorov-Smirnov sobre la posición del efector, la posición de la pieza y su orientación quedan entre 0,232 y 0,696 en valor p , sin que de ello se concluya equivalencia. En cambio, el efecto de aprovechar también los 112 episodios descartados no se ha medido y queda recogido en el Apartado 5.2.

La procedencia pública del conjunto no sustituye a la comprobación del archivo concreto que se ha utilizado. La Tabla 4 recoge los controles de integridad aplicados sobre él, junto con la huella que lo identifica de forma unívoca.

Conviene precisar, por último, que la evaluación no consume datos de este fichero. El rendimiento se mide ejecutando la política en el simulador sobre condiciones iniciales generadas de forma independiente, según se detalla en el Apartado 3.6.

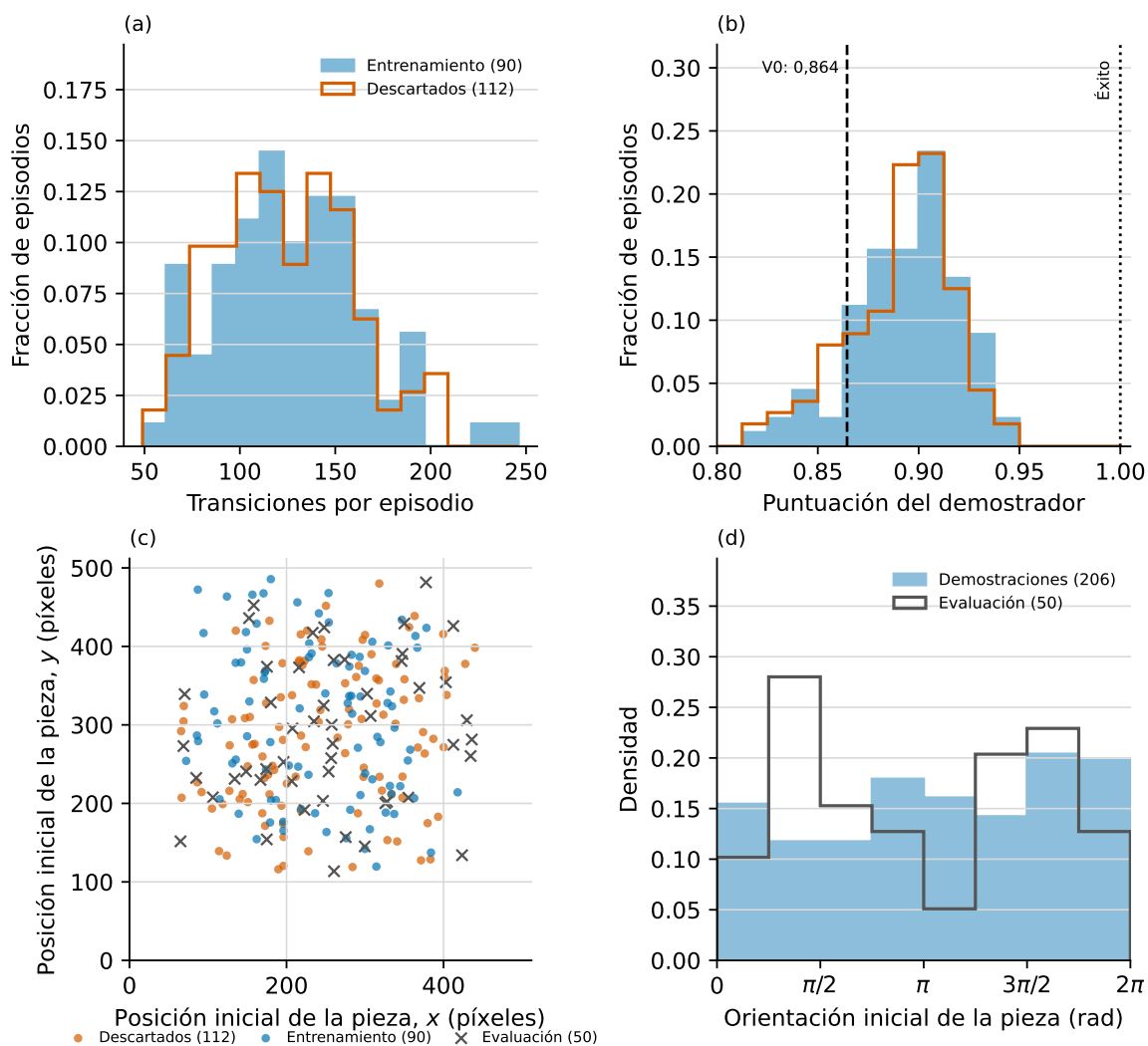


Figura 1. Caracterización del conjunto de demostraciones de Push-T: (a) longitud de los episodios; (b) puntuación del demostrador, con el máximo de V0 y el umbral de éxito; (c) posición inicial de la pieza, con las 50 condiciones de evaluación superpuestas; (d) orientación inicial de la pieza.

Fuente: elaboración propia.

Tabla 4. Controles de integridad del archivo de demostraciones.

Control	Regla	Resultado
Valores ausentes	Ningún NaN en los cinco arreglos	0
Valores infinitos	Ningún Inf en los cinco arreglos	0
Longitud coherente	Todos los arreglos con 25.650 filas	Cumple
Índices de episodio	Estrictamente crecientes, 206 finales	Cumple
Longitud de episodio	Entre 49 y 246 transiciones	Cumple
Rango de la acción	Coordenadas en [12; 511] píxeles	Cumple
Rango del estado	Coordenadas en [0,0002; 510,96]	Cumple
Rango de la imagen	Intensidades en [65; 255], float32	Cumple
Estados duplicados	Filas de state repetidas	0
Tamaño en disco	Suma de los bloques del directorio	31,0 MB

Los controles se ejecutan sobre el propio archivo `zarr`, abierto en modo lectura y recorrido por bloques para no cargar en memoria los 2,84 GB del arreglo de imágenes. La huella SHA-256 del árbol de ficheros, calculada sobre su contenido en orden determinista, es `c235ab79...275e36a4`; el valor completo figura en la Tabla 11.

Fuente: elaboración propia.

3.3 Arquitectura de la política

Diffusion Policy modela la distribución condicional $p(\mathbf{A}_t \mid \mathbf{O}_t)$, donde $\mathbf{O}_t$ agrupa las observaciones de los últimos T_o instantes y $\mathbf{A}_t$ es una secuencia de T_p acciones futuras [3]. El muestreo de esa distribución se realiza mediante un modelo probabilístico de difusión con eliminación de ruido (*Denoising Diffusion Probabilistic Model*, DDPM) [4].

El planificador de ruido fija una sucesión $\beta_1, \dots, \beta_K$ y, a partir de ella, $\alpha_k = 1 - \beta_k$ y $\bar{\alpha}_k = \prod_{j=1}^k \alpha_j$. El proceso directo perturba una secuencia de acciones real $\mathbf{A}_t^0$ interpolando entre la señal limpia y ruido gaussiano $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, con los coeficientes que recoge la Ecuación (1) del Capítulo 2:

$$\mathbf{A}_t^k = \sqrt{\bar{\alpha}_k} \mathbf{A}_t^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon. \quad (3)$$

La red ϵ_θ se ajusta para predecir ese ruido a partir del estado perturbado, del índice de paso y de la observación. La función de pérdida es el error cuadrático medio sobre el ruido, recogido en la Ecuación (4).

$$\mathcal{L} = \mathbb{E}_{k, \epsilon} \left[\left\| \epsilon - \epsilon_\theta \left(\mathbf{O}_t, \sqrt{\bar{\alpha}_k} \mathbf{A}_t^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon, k \right) \right\|^2 \right] \quad (4)$$

En la Ecuación (4), el índice k se muestrea de forma uniforme en $\{1, \dots, K\}$ y determina, a través de $\bar{\alpha}_k$, la magnitud del ruido inyectado. La observación $\mathbf{O}_t$ actúa como condicionamiento, no como variable generada, lo que reduce el coste de inferencia. Por tanto, la calidad de la representación visual condiciona directamente lo que la red generativa puede aprender.

Durante la inferencia, la política parte de una secuencia de ruido puro $\mathbf{A}_t^K \sim \mathcal{N}(0, \mathbf{I})$ y aplica K transiciones inversas según la Ecuación (5).

$$\mathbf{A}_t^{k-1} = \frac{1}{\sqrt{\alpha_k}} \left(\mathbf{A}_t^k - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_\theta \left(\mathbf{O}_t, \mathbf{A}_t^k, k \right) \right) + \sigma_k \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}) \quad (5)$$

La Ecuación (5) es la forma concreta que adoptan los coeficientes genéricos α , γ y σ del muestreo DDPM: $\alpha = \alpha_k^{-1/2}$, $\gamma = \beta_k(1 - \bar{\alpha}_k)^{-1/2}$ y, con la varianza fija menor que emplea la configuración de este trabajo, $\sigma_k^2 = \frac{1-\bar{\alpha}_{k-1}}{1-\bar{\alpha}_k} \beta_k$. El término $\mathbf{z}$ se anula en el último paso, $k = 1$. El ruido fresco que se añade en los demás preserva el carácter estocástico del muestreo y, con ello, la capacidad de representar varios modos de acción.

El planificador concreto es el DDPM con $K = 100$ pasos, idéntico en entrenamiento y en inferencia, sin acelerarlo mediante muestreadores implícitos. Su sucesión de β_k sigue el plan de coseno al cuadrado, definido por $\bar{\alpha}_k = f(k)/f(0)$ con $f(k) = \cos^2\left(\frac{k/K+s}{1+s} \cdot \frac{\pi}{2}\right)$ y $s = 0,008$, y los β_k resultantes se acotan superiormente en 0,999 para evitar la singularidad final. La red predice el ruido y no la muestra limpia, y la estimación intermedia de $\mathbf{A}_t^0$ se recorta al intervalo $[-1, 1]$ en el que se normalizan las acciones. Con este plan, los parámetros de inicio y fin de una progresión lineal de β no intervienen.

Las acciones generadas se aplican bajo un esquema de horizonte deslizante. La política recibe $T_o = 2$ observaciones, predice $T_p = 16$ acciones y ejecuta únicamente las $T_a = 8$ primeras antes de replanificar [3]. Esta configuración equilibra la consistencia temporal de la trayectoria con la capacidad de reaccionar ante cambios en la escena.

La red ϵ_θ es una red convolucional unidimensional con arquitectura en U y dimensiones de canal 512, 1.024 y 2.048 en sus tres niveles de submuestreo. El vector de observación se inyecta como condicionamiento global en cada capa mediante modulación lineal por características (*Feature-wise Linear Modulation*, FiLM) [24]. Este bloque concentra unos 277 millones de parámetros y, por tanto, domina el coste de entrenamiento.

Delante de la red generativa opera el codificador de observaciones. Dicho módulo aplica las transformaciones de imagen propias de cada arquitectura, extrae un vector de características visuales y lo concatena con el estado del efector. La configuración de referencia empleada aquí utiliza una ResNet-18 [15] entrenada desde cero cuya capa de clasificación se sustituye por la identidad, de modo que la agregación espacial es el promediado global de la propia red. La única modificación estructural consiste en reemplazar la normalización por lotes por normalización por grupos [25], ya que la primera interfiere con la media móvil exponencial de los pesos.

Conviene precisarlo porque el artículo original describe además un *spatial softmax* que conserva la localización de las activaciones [26]. Esa variante corresponde al codificador basado en *robomimic* y no a la configuración de Push-T con imágenes que este trabajo reproduce, donde el módulo codificador aplica únicamente redimensionado, recorte y normalización antes de la red convolucional. Ninguna de las cinco variantes de este estudio emplea, por tanto, *spatial softmax*: V0, V1 y V2 agregan por promediado global y V3 y V4 toman el componente de clase del transformador.

3.4 Variantes del codificador visual

Se definen cinco variantes, identificadas de V0 a V4, que modifican el bloque visual y el tratamiento de sus pesos. La Tabla 5 resume su configuración. Las tres primeras comparten arquitectura convolucional y permiten contrastar estrategias de entrenamiento; las dos últimas introducen transformadores de visión preentrenados con objetivos distintos.

V0: línea base entrenada desde cero

Tabla 5. Configuración de las cinco variantes del codificador visual evaluadas.

Variante	Codificador visual	Pesos iniciales	Estrategia	Parámetros (millones)	
				Totales	Entrenables
V0	ResNet-18	Aleatorios	Desde cero	288	288
V1	ResNet-18	ImageNet-1k	Congelado	288	277
V2	ResNet-18	ImageNet-1k	Ajuste fino	288	288
V3	DINOv2 ViT-S/14	LVD-142M	Congelado	299	277
V4	CLIP ViT-B/16	WIT-400M	Congelado	363	277

Fuente: elaboración propia.

La variante V0 reproduce la configuración original de *Diffusion Policy* [3]. Emplea una ResNet-18 con pesos aleatorios, opera sobre la imagen a su resolución nativa de 96 píxeles y aplica recorte aleatorio como único aumento de datos. El codificador se entrena de extremo a extremo junto con la red generativa y constituye la referencia frente a la que se comparan las demás variantes.

V1 y V2: transferencia desde ImageNet

Las variantes V1 y V2 parten de una ResNet-18 preentrenada sobre ImageNet-1k [16] y difieren en el tratamiento de sus pesos. En V1 el codificador permanece congelado, de modo que actúa como extractor fijo de características; en V2 sus pesos se actualizan junto con el resto de la política. Ambas redimensionan la imagen a 224 píxeles por interpolación bilineal y aplican la normalización estándar de ImageNet, con idénticas constantes de media y desviación típica.

Sin embargo, **V1 y V2 no parten del mismo archivo de pesos**, y el contraste entre ambas no puede leerse como el efecto aislado de la estrategia de entrenamiento. V1 carga el modelo `resnet18` de `timm` con su etiqueta por defecto, `resnet18.a1_in1k`, obtenida con la receta A1 de *ResNet strikes back* [27]; V2 carga el modelo `resnet18` de `torchvision` con la enumeración `ResNet18_Weights.IMAGENET1K_V1`, que corresponde a la receta original de la biblioteca. Ambos son ResNet-18 ajustadas sobre ImageNet-1k, pero proceden de procedimientos de entrenamiento distintos. La comprobación es directa: como el codificador de V1 permanece congelado, los 120 tensores de su punto de control deben coincidir con los pesos de partida. Comparados tensor a tensor, coinciden de forma exacta con `resnet18.a1_in1k` y no coincide ninguno con `IMAGENET1K_V1`. El Apartado 3.9 recoge el procedimiento, su resultado y los identificadores completos de ambos archivos de pesos.

En consecuencia, la diferencia observada entre V1 y V2 combina la estrategia de adaptación con el origen de los pesos, y este trabajo no la atribuye a la primera. Un diseño que aislara la estrategia debería inicializar ambas desde el mismo archivo y duplicar el estado antes de congelar una de las copias; queda descrito en el Apartado 5.2.

El contraste entre V0 y las dos variantes preentrenadas tampoco aísla el efecto de la inicialización, ya que el preprocesado y la configuración interna viajan con los pesos. La Tabla 6 detalla la cadena completa de cada variante. Entre V0 y V2 varían de forma conjunta tres factores: los pesos iniciales, la resolución efectiva de entrada junto con el recorte aleatorio y la capa de normalización interna de la red, que en V0 es por grupos y en V1 a V4 por lotes. Entre V0 y V1 se añade un cuarto, el entrenamiento conjunto del codificador frente a su congelación; y frente a V3 y V4 cambia además la arquitectura. Por tanto,

esas comparaciones deben leerse sobre el bloque visual completo y no sobre ninguno de sus factores por separado. Los entrenamientos que los separarían se describen en el Apartado 5.2.

Tabla 6. Cadena de preprocesado de la imagen por variante, desde el fotograma renderizado hasta el tensor que recibe la red.

Variante	Entrada	Redimensionado	Recorte	Normalización	Tensor final
V0	96×96	—	76×76 aleatorio*	ImageNet	76×76
V1	96×96	224×224 bilineal	—	ImageNet	224×224
V2	96×96	224×224 bilineal	—	ImageNet	224×224
V3	96×96	224×224 bilineal	—	ImageNet	224×224
V4	96×96	224×224 bilineal	—	CLIP	224×224

Fuente: elaboración propia.

Tres precisiones sobre la Tabla 6. Primera, el recorte de V0 es aleatorio durante el entrenamiento y central en evaluación, de modo que el tensor mide siempre 76×76 y el aumento de datos solo actúa en el ajuste; es la única variante con aumento. Segunda, la normalización ImageNet emplea las mismas constantes en las cuatro variantes que la usan, media (0,485; 0,456; 0,406) y desviación típica (0,229; 0,224; 0,225), aunque en V0 y V2 la aplica el módulo codificador y en V1 y V3 la aplica el propio envoltorio del modelo preentrenado. Tercera, la normalización de V4 emplea las constantes de CLIP, media (0,481; 0,458; 0,408) y desviación típica (0,269; 0,261; 0,276), tal como exigen sus pesos.

V3 y V4: transformadores de visión preentrenados

Las variantes V3 y V4 sustituyen la red convolucional por un transformador de visión [17]. V3 utiliza DINOv2 ViT-S/14, preentrenado de forma autosupervisada sobre el corpus LVD-142M [18]. V4 utiliza CLIP ViT-B/16, preentrenado con supervisión de lenguaje natural sobre pares de imagen y texto [19]. En ambos casos los pesos permanecen congelados, decisión que impone la memoria gráfica disponible. La medida recogida en la Tabla 18 la respalda: el ajuste fino de la ResNet-18 de V2, con once millones de parámetros en el codificador, ya reclama más memoria de la que tiene la tarjeta y solo se completa porque el controlador respalda el exceso con memoria del sistema, a costa de triplicar el tiempo por época. Los dos transformadores aportan 22 y 86 millones de parámetros a ese bloque, según se deduce de la Tabla 5, de modo que ajustarlos agravaría el problema en lugar de aliviarlo.

Dado que la dimensión de salida de estos modelos no coincide con la de la ResNet-18, se añade una capa lineal de proyección que reduce el descriptor a 512 componentes. Dicha capa sí se entrena, puesto que forma parte de la política y no del modelo preentrenado. Los codificadores preentrenados se cargan mediante la biblioteca `timm` [28], salvo la ResNet-18 de V2, que procede de `torchvision`.

3.5 Protocolo de entrenamiento

Las variantes comparten los hiperparámetros recogidos en la Tabla 7, salvo las dos excepciones indicadas en ella. El presupuesto se redujo durante la campaña conforme aumentaba el coste observado: 500 épocas para V0 y V1, 300 para V2 y V3, y 200 para V4. Estos valores quedan muy por debajo de las 3.000 épocas del artículo original. Dentro de cada presupuesto

se aplica el criterio de parada que se precisa más adelante.

Tabla 7. Hiperparámetros de entrenamiento de las cinco variantes.

Parámetro	Valor
Horizonte de predicción T_p	16 pasos
Pasos de observación T_o	2
Pasos de acción ejecutados T_a	8
Pasos de difusión (entrenamiento e inferencia)	100
Planificador de ruido	DDPM, varianza fija, predicción de ruido
Optimizador	AdamW [29]
Tasa de aprendizaje	1×10^{-4} , decaimiento coseno
Pasos de calentamiento	500
Media móvil exponencial de los pesos	Activada
Tamaño de lote	64 (32 en V3 y V4)
Acumulación de gradiente	1 (2 en V3 y V4)
Tamaño de lote efectivo	64 en las cinco variantes
Presupuesto máximo de épocas	500 (V0–V1); 300 (V2–V3); 200 (V4)
Semilla	42

Fuente: elaboración propia.

Además del presupuesto, el tamaño de lote de V3 y V4 se redujo a la mitad, ya que los dos transformadores operan a 224 píxeles y comprometen la memoria gráfica disponible con lotes de 64 muestras. La reducción se compensa acumulando el gradiente durante dos lotes antes de cada actualización, de modo que el tamaño de lote efectivo y el número de actualizaciones por época coinciden con los de V0, V1 y V2: 336 lotes y 168 actualizaciones frente a 168 lotes y otras tantas actualizaciones. Lo que sí cambia es el número de pasadas por el codificador, que se duplica, y con él el tiempo por época. La Tabla 18 matiza esta decisión: con lote reducido V4 llega al 90,3 % de la memoria de la tarjeta, de modo que la reducción estaba justificada, mientras que V3 se queda en el 78,2 % y disponía de más margen. En V3 la medida no sostiene la necesidad y la reducción debe leerse como cautela y como forma de igualar las condiciones de los dos transformadores.

Cada entrenamiento se lanza mediante un script de control que fija los parámetros de ejecución, verifica la memoria disponible e impide que se ejecuten dos procesos de forma simultánea. Esta salvaguarda responde a un incidente previo, en el que dos procesos concurrentes sobre el mismo directorio de salida agotaron la memoria del sistema y corrompieron el registro de métricas.

La Tabla 8 recoge la configuración que Hydra resolvió efectivamente para cada ejecución, tomada de los ficheros que el propio marco de configuración escribe junto a los resultados. Se listan solo los parámetros que difieren entre variantes; los comunes se enuncian bajo la tabla.

Durante el entrenamiento se guarda un punto de control cada 50 épocas y se conservan los tres mejores según la puntuación de evaluación. El modelo evaluado en el Capítulo 4 es el de mayor puntuación, criterio coherente con el empleado en la literatura de la tarea [8].

El criterio de parada anticipada se apoya en esos mismos puntos de control. Un entrenamiento se interrumpe cuando la puntuación de evaluación no supera su máximo en dos evaluaciones consecutivas, esto es, tras 100 épocas sin mejora, mientras la pérdida de entrenamiento sigue descendiendo. Esa combinación apunta a sobreajuste, de modo que

Tabla 8. Configuración efectiva de las cinco ejecuciones. Solo se muestran los parámetros que difieren entre ellas.

Parámetro	V0	V1	V2	V3	V4
Presupuesto de épocas	500	500	300	300	200
Épocas completadas	500	500	266	155	200
Tamaño de lote	64	64	64	32	32
Acumulación de gradiente	1	1	1	2	2
Lote efectivo	64	64	64	64	64
Actualizaciones por época	168	168	168	168	168
Codificador congelado	No	Sí	No	Sí	Sí
Normalización de la red	Grupos	Lotes	Lotes	Capas	Capas
Guardado cada (épocas)	50	50	50	10	10

Parámetros comunes a las cinco ejecuciones: semilla 42; optimizador AdamW [29] con tasa de aprendizaje 1×10^{-4} , momentos (0,95; 0,999), $\varepsilon = 1 \times 10^{-8}$ y decaimiento de pesos 1×10^{-6} ; planificador de tasa de coseno con 500 pasos de calentamiento; media móvil exponencial con potencia 0,75 y valor máximo 0,9999; validación cada época; evaluación en el simulador cada 50 épocas con 8 procesos; y conservación de los tres mejores puntos de control según la puntuación de selección, además del último. Las configuraciones completas y los registros de cada ejecución residen en el repositorio del trabajo [30].

Fuente: elaboración propia.

prolongar la ejecución no aportaría un punto de control mejor. La parada no se aplica antes de la época 200, para no interrumpir una variante de convergencia lenta. El criterio no altera el modelo seleccionado, puesto que este se elige entre los puntos de control ya guardados.

Este criterio no precedió al experimento, sino que se adoptó durante su ejecución. Las dos primeras variantes, V0 y V1, agotaron 500 épocas porque entonces no existía regla de parada. La formulación anterior se fijó al observar sus curvas y se aplicó a V2, detenida tras 266 épocas completas. V3 se interrumpió manualmente tras 155 épocas completas, es decir en la de índice 154, antes del mínimo de 200 que la propia regla fijaba, mientras que V4 agotó sus 200 épocas. Los índices de época empiezan en cero, de modo que ambas cifras designan el mismo instante; en el resto de la memoria se emplea siempre el recuento de épocas completas. Las decisiones se tomaron por inspección, no de forma automática.

Esta asimetría afecta al coste y a la selección del modelo. V3 y V4 solo disponen de cuatro evaluaciones, frente a las diez de V0 y V1, por lo que tuvieron menos oportunidades de alcanzar un máximo tardío. El Apartado 3.10 reconstruye el contrafactual bajo el criterio aplicado de forma uniforme, dentro del presupuesto asignado a cada variante.

3.6 Protocolo de evaluación

La evaluación se ejecuta cada 50 épocas sobre 56 condiciones iniciales del simulador. Seis de ellas proceden de la distribución de entrenamiento y las 50 restantes se generan con semillas de entorno reservadas, no empleadas durante el ajuste. Las primeras permiten estimar el ajuste a los datos vistos; las segundas constituyen la medida de rendimiento que se reporta.

Cada condición inicial se resuelve en un episodio completo de bucle cerrado. La política recibe las observaciones, genera una secuencia de acciones, ejecuta las primeras T_a y repite el ciclo hasta alcanzar el límite de pasos del episodio o completar la tarea. Los pesos

utilizados son los de la media móvil exponencial, no los del modelo instantáneo, tal como prescribe la implementación de referencia.

Las condiciones se agrupan en tandas de ocho procesos para no agotar la memoria del sistema. Esta agrupación modifica el tiempo de ejecución de la evaluación, pero no las métricas resultantes, puesto que la agregación recorre las 56 condiciones con independencia del número de procesos simultáneos.

3.7 Métricas

La métrica principal es la puntuación de cobertura de la tarea Push-T [3]. Para un episodio j , se calcula en cada instante t la cobertura $c_{j,t}$, definida como la fracción del área objetivo cubierta por la pieza. La puntuación del episodio se obtiene según la Ecuación (6).

$$s_j = \max_{1 \leq t \leq T} \min\left(\frac{c_{j,t}}{\tau}, 1\right) \quad (6)$$

En la Ecuación (6), T es la duración máxima del episodio y τ el umbral de cobertura que define el éxito, fijado en 0,95 por la implementación de referencia. La puntuación resulta así continua y acotada en el intervalo unidad: vale uno cuando la pieza alcanza el umbral en algún instante y crece de forma proporcional a la cobertura en caso contrario. Se toma el máximo a lo largo del episodio, y no el valor final, porque la pieza puede desplazarse tras alcanzar la posición objetivo.

La métrica que se reporta es la media de s_j sobre los 50 episodios evaluados, denominada puntuación media de evaluación. Junto a ella se reportan la desviación típica y el error estándar de esos 50 valores, magnitudes que describen la dispersión de la propia métrica. Se registran además cuatro cantidades auxiliares: la pérdida de entrenamiento, la pérdida de validación, el error cuadrático medio de la acción predicha y la puntuación media sobre las seis condiciones que la implementación de referencia denomina de entrenamiento. Las dos primeras describen el ajuste y su comparación es la que revela el sobreajuste.

Esa cuarta cantidad exige una advertencia. Sus seis condiciones iniciales son las de las demostraciones 0 a 5, y el sorteo descrito en el Apartado 3.2 dejó las seis fuera del subconjunto de entrenamiento. La puntuación que se obtiene sobre ellas no mide, por tanto, el rendimiento en condiciones vistas durante el ajuste, y no se emplea en esta memoria como indicio de sobreajuste.

El nombre de la métrica principal requiere una precisión. Las 50 condiciones se generan con semillas reservadas y no intervienen en el ajuste de los pesos, pero se evalúan cada 50 épocas y determinan qué punto de control se conserva. Cumplen, por tanto, la función de un conjunto de selección y no la de una evaluación final independiente. El término *prueba* se reserva para la medida sobre el bloque disjunto que describe el Apartado 3.8, donde se detallan también el estimando, las unidades experimentales y el tratamiento estadístico de las diferencias.

El coste computacional se caracteriza mediante cuatro indicadores. El primero es el número de parámetros totales y entrenables de cada variante, recogido en la Tabla 5. El segundo es el tiempo por paso de optimización, medido tras la fase de calentamiento y agregado por época en la Tabla 12. El tercero es la latencia de inferencia para un lote de tamaño fijo. El

cuarto es el pico de memoria gráfica reservada durante el entrenamiento. Los dos últimos se reportan en el Apartado 4.5.

La latencia se desglosa en tres magnitudes, porque una sola oculta el efecto que se estudia. Cada llamada a la política ejecuta el codificador visual una vez y la red generativa cien veces, de modo que el tiempo total lo domina un componente idéntico en las cinco variantes. Se reportan, por tanto, la latencia de la llamada completa, la del codificador aislado y la de la llamada dividida entre las ocho acciones que emite, esta última porque acota la frecuencia de control alcanzable. Las medidas se toman en precisión `float32`, con lotes de 1 y 8 muestras, y se resumen mediante la mediana y el recorrido intercuartílico de 50 repeticiones cronometradas, precedidas de 20 descartadas. Las repeticiones se alternan entre variantes por rondas, para que la deriva térmica del equipo portátil afecte por igual a todas.

El pico de memoria exige reproducir el paso de optimización completo, ya que los gradientes, los momentos de AdamW y la copia de la media móvil son justamente lo que ocupa la tarjeta. Se ejecutan doce pasos sobre lotes reales, con el tamaño propio de cada variante, y se registra la memoria reservada por el asignador. Se acompaña del pico en inferencia, que describe lo que exigiría un despliegue.

Ambas medidas se toman en entornos de ejecución distintos y esa diferencia se declara en cada tabla. La latencia se mide en el entorno de inferencia sobre Windows y el pico de memoria en el entorno de WSL en el que se entrenó, puesto que el asignador de memoria de PyTorch cambió entre ambas versiones y una cifra tomada en Windows no describiría las ejecuciones recogidas en la Tabla 12. Dentro de cada indicador, las cinco variantes comparten entorno.

3.8 Selección del punto de control y prueba final

Los conjuntos de condiciones iniciales cumplen dos funciones distintas que conviene no confundir. El primero elige qué punto de control representa a cada variante; el segundo mide su rendimiento sin haber intervenido en esa elección. La Figura 2 resume el recorrido completo, desde los episodios de demostración hasta la tabla de resultados finales.

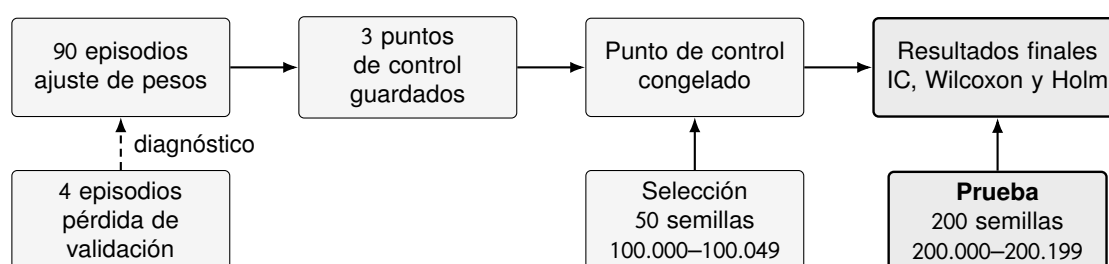


Figura 2. Flujo de ajuste, validación, selección y prueba. Las 50 semillas de selección solo alcanzan la tabla final a través del punto de control que congelan. Los 112 episodios restantes del conjunto de datos no intervienen en ninguna etapa.

Fuente: elaboración propia.

Estimando y unidades experimentales

El estimando primario es la puntuación media de cobertura de un punto de control fijo sobre el generador de condiciones iniciales de Push-T. Se estima con las 200 condiciones

del bloque de prueba, muestreadas de ese generador mediante semillas consecutivas. La variable de resultado secundaria es la tasa de éxito, es decir, la fracción de condiciones cuya puntuación alcanza el umbral.

El estudio maneja tres unidades distintas y solo replica una de ellas. El episodio de demostración construye las ventanas de entrenamiento; la condición inicial evalúa un punto de control ya fijado; la ejecución completa de entrenamiento permitiría comparar estrategias. Existen 90, 200 y 1 unidades por variante en esos tres niveles. Por tanto, la unidad de replicación de este trabajo es la condición inicial: las cifras estiman el rendimiento de cinco artefactos concretos, no el rendimiento esperado de volver a entrenar cada estrategia. Esa restricción se mantiene en las conclusiones y se recoge en el Apartado 4.8.

Bloque de prueba disjunto

La prueba final emplea las semillas de entorno 200.000 a 200.199. El bloque es disjunto de las semillas 0 a 205 que generaron las demostraciones y de las 50 de selección, condición que el programa de evaluación comprueba antes de ejecutar nada. El desplazamiento a 200.000, en lugar de continuar la serie de selección, evita cualquier ambigüedad al leer las tablas.

Se evalúa una sola vez cada uno de los cinco puntos de control ya seleccionados, con los pesos de la media móvil exponencial y la misma configuración del simulador que durante el entrenamiento. Ninguna decisión posterior a esa medida modifica qué punto de control representa a cada variante. El protocolo, incluidas las semillas, el tamaño de muestra, las variables de resultado y el análisis, se fijó por escrito y se registró en el repositorio antes de ejecutar la evaluación.

Conviene declarar el límite de este procedimiento. Durante el entrenamiento solo se conservaron los tres mejores puntos de control de cada variante según la puntuación de selección, de modo que el conjunto de candidatos ya viene filtrado por esa métrica. El bloque disjunto acota el sesgo de selección, pero no lo elimina por completo.

Ruido de difusión sincronizado

La política parte de ruido y añade ruido en cada uno de los 100 pasos de difusión, según la Ecuación (5). Dos variantes evaluadas en la misma condición inicial no formarían un par comparable si cada una recibiera una realización distinta de ese ruido. Para evitarlo se aplica la técnica de los números aleatorios comunes: el generador se siembra antes de cada llamada a la política en función de la posición del ciclo de control, con una semilla base común a las cinco variantes.

El esquema produce ruido idéntico entre variantes porque todas comparten la forma del tensor de ruido, el número de tandas y el número de llamadas por tanda. En modo evaluación, además, el recorte aleatorio de V_0 pasa a ser un recorte central, de modo que el codificador no consume números aleatorios. Cada condición se resuelve con una sola trayectoria de difusión, compartida por las cinco variantes, y el estimando queda definido en consecuencia.

Especificación estadística

Las diferencias se calculan por pares, emparejando cada variante con las demás sobre la misma condición inicial. Cada diferencia media se acompaña de un intervalo de confianza del 95 % obtenido por *bootstrap* BCa con 10.000 remuestreos de los pares y semilla fija. Este

método se elige porque no supone normalidad de las diferencias y corrige el sesgo y la asimetría de la distribución remuestreada.

El contraste preregistrado es la prueba de los rangos con signo de Wilcoxon. Los pares nulos se descartan según el criterio del propio Wilcoxon. Con 200 pares la distribución exacta no resulta aplicable, de modo que se usa la aproximación normal con corrección de continuidad; el método empleado se registra en cada fila de la tabla de contrastes.

A ese contraste se añade una prueba de permutación sobre la diferencia media, por inversión de signo de los pares, con 10.000 permutaciones y semilla fija. Su inclusión responde a una objeción pertinente: el estimando primario declarado es una diferencia de medias, y Wilcoxon opera sobre los rangos de las diferencias, no sobre su media. La prueba de permutación toma como estadístico la propia media y contrasta la simetría de las diferencias en torno a cero, de modo que apunta a la cantidad que la memoria reporta. Se declara sin ambigüedad como **análisis post hoc**: no figuraba en el protocolo preregistrado y se añadió después de observar el bloque de prueba. Ambos contrastes se reportan juntos en el Apartado 4.3 y ninguno sustituye al otro; donde discrepan, la memoria adopta la lectura más conservadora.

La familia de hipótesis la forman las diez comparaciones por pares de la variable de resultado principal, y cada contraste se corrige sobre su propia familia de diez. La tasa de éxito es una variable secundaria y descriptiva: se acompaña de intervalos de Wilson y no entra en ninguna familia corregida ni sostiene conclusión alguna por sí sola. Sus valores p se ajustan mediante el procedimiento secuencial de Holm, que controla el error familiar sin suponer independencia entre contrastes, y la significación se determina sobre los valores ajustados con un umbral de 0,05. Los intervalos de confianza no incorporan ese ajuste y se presentan como descriptivos.

Estas medidas caracterizan la variabilidad debida a las condiciones iniciales y, gracias a la sincronización del ruido, no la mezclan con la del muestreo de difusión. No sustituyen, en cambio, a la variabilidad entre semillas de entrenamiento, que exigiría repetir los ajustes y queda fuera del alcance fijado en el Apartado 1.4.

3.9 Entorno de ejecución y reproducibilidad

Todos los experimentos se ejecutan sobre un único equipo portátil, cuyas características se recogen en la Tabla 9. El cómputo no se reparte entre varias máquinas ni se recurre a servicios en la nube, de modo que las medidas de coste del Apartado 3.10 son directamente comparables entre variantes.

La restricción determinante es la memoria gráfica de ocho gigabytes. Dicho límite condiciona el tamaño de lote de V4 y descarta el ajuste fino de los transformadores de visión, según se justificó en el Apartado 3.4. La memoria principal impone una segunda restricción menos evidente: el conjunto de datos se carga íntegramente en RAM y ocupa unos 2,8 GB por proceso, cifra que obliga a limitar los procesos de carga y de simulación. La asignación de memoria al subsistema se amplió de 13 a 18 GB antes de ejecutar V2, ya que el ajuste fino del codificador aumenta el consumo respecto a las variantes congeladas.

Sobre esa plataforma se despliega la pila de software detallada en la Tabla 10. Las versiones están fijadas y no se actualizan entre variantes, puesto que la comparación exige que el único factor que cambie sea el codificador visual.

Tabla 9. Plataforma de ejecución de los experimentos.

Bloque	Componente	Características
Cómputo	Procesador	Intel Core i9-12900H, 16 núcleos lógicos
	GPU	NVIDIA GeForce RTX 3070 Ti Laptop, 8 GB
	Capacidad CUDA	8.6 (Ampere), controlador 596.21
Memoria	RAM física	23,7 GB
	RAM del subsistema	18 GB, más 16 GB de intercambio
Plataforma	Sistema anfitrión	Windows 11
	Subsistema	WSL 2.5.9.0, núcleo 6.6.87.2
	Distribución	Ubuntu 24.04.2 LTS
Almacenamiento	Disco de trabajo	1.007 GB virtuales (ext4 sobre VHDX)

Fuente: elaboración propia.

Tabla 10. Componentes de la pila de software y función de cada uno.

Componente	Versión	Función en el experimento
Python	3.9.15	Intérprete, en entorno conda aislado
PyTorch	1.12.1	Marco de trabajo de aprendizaje profundo [31]
CUDA	11.6	Biblioteca de cálculo sobre GPU
torchvision	0.13.1	ResNet-18 y pesos de ImageNet-1k
timm	0.9.16	DINOv2 y CLIP preentrenados [28]
diffusers	0.11.1	Planificador de ruido DDPM
huggingface_hub	0.25.2	Descarga de los pesos preentrenados
robomimic	0.2.0	Utilidades de clonación de comportamiento [8]
diffusion_policy	Bifurcación propia	Política, entorno Push-T y evaluación
Hydra	1.2.0	Configuración y registro de cada ejecución
Zarr	2.12.0	Formato del conjunto de demostraciones

Fuente: elaboración propia.

El código de partida es la implementación pública de *Diffusion Policy* [3], sobre la que se añaden los codificadores de V1 a V4 y sus configuraciones. Las modificaciones se concentran en el módulo del codificador de observaciones, mientras que la red generativa, el bucle de entrenamiento y el evaluador permanecen sin cambios. Por tanto, cualquier diferencia entre variantes es atribuible al codificador y no a la infraestructura que lo rodea. El código modificado, las configuraciones de las cinco variantes, los guiones de análisis y el protocolo de la prueba final residen en el repositorio del trabajo [30]. El desarrollo matemático detallado del método se recoge en una monografía propia previa [32].

Identificadores exactos de los artefactos

Reproducir estas variantes exige el identificador preciso de cada archivo de pesos, y no solo el nombre de la arquitectura: dos recetas distintas de entrenamiento sobre ImageNet-1k producen ResNet-18 diferentes. La Tabla 11 los recoge, junto con la huella SHA-256 de cada punto de control evaluado y la del conjunto de demostraciones.

Tabla 11. Identificadores de los pesos de partida y huellas de los artefactos evaluados.

Variante	Pesos de partida	SHA-256 del punto de control evaluado
V0	<code>torchvision:resnet18, weights=None</code>	5310551ee71075d9efcf956c34670809741d84e06808809551e7675674e8ce63
V1	<code>timm:resnet18.a1_in1k</code>	bb5012c206c2631ff8960592060eb0409244c9b9319172bd28ed720b0e7c175b
V2	<code>torchvision:ResNet18_Weights.IMAGENET1K_V1</code>	36deb633c82033d81ed6b7fb1b16dcbfd0bf42c1dc7fb20b5dfe784fd357eff5
V3	<code>timm:vit_small_patch14_dinov2</code>	1403e6ba251ac818f3ce1d7a8faf50048517b32c87f3efdf43f751c74f1a412c
V4	<code>timm:vit_base_patch16_clip_224</code>	2fcd5857bcbb1417e9e6f159b091b8bffd948e0bb64ab28791502c679bca8b0
Conjunto de demostraciones (árbol zarr)		c235ab79adceeada3de959baae1bbd37ab8d49fbc4612779bad837ba275e36a4

Los puntos de control corresponden a las épocas 350, 150, 150, 100 y 100. En V1, V3 y V4 la configuración solicita el modelo sin etiqueta explícita y `timm` resuelve la etiqueta por defecto; para `resnet18` esa etiqueta es `a1_in1k`, de la receta A1 de *ResNet strikes back* [27], distinta de la receta original de `torchvision` que emplea V2. La huella del conjunto de demostraciones se calcula sobre el contenido de sus ficheros en orden determinista, puesto que un `zarr` es un directorio y no un fichero único.

Fuente: elaboración propia.

La afirmación del Apartado 3.4 sobre la distinta procedencia de los pesos de V1 y V2 se comprueba, y no se supone. Como el codificador de V1 permanece congelado durante todo el entrenamiento, los 120 tensores de su punto de control deben coincidir con los pesos de partida. Comparados uno a uno, coinciden con `resnet18.a1_in1k` en los 120 y con diferencia máxima nula, mientras que frente a `IMAGENET1K_V1` no coincide ninguno y la mayor discrepancia supera las $3,7 \times 10^5$ unidades, magnitud incompatible con un error de redondeo.

Preespecificación del protocolo

El protocolo de la prueba final del Apartado 3.8 se fijó por escrito antes de ejecutarla, y esa anterioridad es verificable. El documento se incorporó al repositorio en la revisión 8bc22e72b15b6ad6987dac37b7ff2aa397af94f1, fechada el 27 de agosto de 2.026 a las 08:31:36, con anterioridad a la ejecución de la evaluación. La marca temporal de esa revisión, y no la declaración del autor, es lo que acredita la preespecificación.

Medidas y límites de la reproducibilidad

La reproducibilidad se apoya en cuatro medidas. En primer lugar, la semilla global se fija en 42 para todas las variantes y las condiciones de evaluación emplean semillas reservadas y constantes. En segundo lugar, las versiones exactas de las bibliotecas quedan fijadas en la Tabla 10. En tercer lugar, la configuración efectiva que Hydra resuelve para cada ejecución se almacena junto a sus resultados y se conserva en el repositorio. En cuarto lugar, los artefactos quedan identificados por su huella en la Tabla 11.

No obstante, la reproducibilidad no es exacta. Los algoritmos deterministas de la biblioteca de aceleración no están activados, ya que resultan incompatibles con las operaciones convolucionales empleadas. En consecuencia, dos ejecuciones con la misma semilla pueden diferir ligeramente, y las diferencias pequeñas entre variantes deben interpretarse con cautela. El Capítulo 4 aplica ese criterio al comparar las puntuaciones obtenidas.

3.10 Coste temporal de los entrenamientos

El coste de cómputo condiciona el alcance del estudio y merece, por ello, una cuantificación explícita. La Tabla 12 recoge la duración de cada ejecución sobre la plataforma descrita en el Apartado 3.9. Se distinguen dos magnitudes: el tiempo del bucle de optimización, medido sobre las épocas de entrenamiento, y el tiempo total transcurrido, que añade la validación por época, las evaluaciones en el simulador y el guardado de puntos de control.

Tabla 12. Coste temporal de los entrenamientos ejecutados.

Variante	Épocas registradas	Bucle por época	Bucle acumulado	Tiempo total
V0	500	1,6 min	13,0 h	17,4 h
V1	500	5,3 min	43,8 h	96,5 h
V2	266	14,7 min	65,3 h [†]	84,9 h [†]
V3	155	2,3 min	5,9 h	10,9 h
V4	200	4,0 min	13,3 h	27,4 h
Total	1.621		141,3 h	237,1 h

[†] Valor estimado a partir del tiempo medio por época de las épocas efectivamente cronometradas.

Fuente: elaboración propia.

Las cinco variantes suman 237,1 h de cómputo, muy por encima de las 75 horas que cabría proyectar a partir de la primera ejecución. La discrepancia procede de dos factores. En primer lugar, el coste por época crece casi un orden de magnitud entre V0 y V2. En segundo lugar, la validación y las evaluaciones en el simulador amplían el tiempo del bucle de optimización, puesto que los episodios se ejecutan en el procesador.

Las dos últimas columnas no admiten, sin embargo, una lectura directa como medida de coste relativo. V2 acumula menos tiempo total que V1 pese a ser casi tres veces más cara por época, simplemente porque se detuvo antes. La comparación entre variantes debe apoyarse, por tanto, en el tiempo por época, mientras que el tiempo total mide lo que ha consumido el experimento tal como se ejecutó.

Tres aclaraciones acotan la lectura de la tabla. En primer lugar, solo V2 conserva el cronómetro de una parte de sus épocas, de modo que su tiempo acumulado se extrapola a partir de la media; el símbolo [†] marca los valores estimados. En segundo lugar, V2 se ejecutó en dos

sesiones separadas por una interrupción de dos días, que se descuenta del tiempo total. En tercer lugar, V3 se detuvo tras registrar 155 épocas de un presupuesto de 300, mientras que V4 completó las 200 asignadas.

Este coste explica dos decisiones del diseño experimental. La primera es el uso de una única semilla por variante, ya que replicar las cinco condiciones tres veces exigiría del orden de 700 horas de la misma GPU. La segunda es la adopción del criterio de parada anticipada del Apartado 3.5: V1 dedicó unas 70 horas a épocas posteriores a su mejor punto de control, sin obtener ninguna mejora.

La tabla mide el experimento tal como se ejecutó y no un protocolo fijado de antemano. Bajo el criterio uniforme, V0 se habría detenido en la época 300 y V1 en la 250. Las cinco ejecuciones habrían sumado 111,4 h de bucle y 177,3 h totales, frente a 141,3 h y 237,1 h. El punto de control seleccionado solo cambiaría en V0, que conservaría el de la época 200 con 0,8643 en lugar del de la 350 con 0,8645. Ninguna conclusión del Capítulo 4 depende de esa diferencia.

4. RESULTADOS

Este capítulo presenta los resultados de las cinco variantes y los relaciona con las expectativas previas formuladas en el Apartado 3.1. El análisis comienza con la cobertura de las ejecuciones, continúa con la evolución del rendimiento y de las pérdidas, y concluye con una comparación cualitativa y las limitaciones de la evidencia.

4.1 Cobertura de las ejecuciones

La Tabla 13 resume la cobertura de los registros y el mejor valor de la métrica principal. V0 y V1 agotaron 500 épocas, V2 completó 266, V3 registró 155 y V4 agotó su presupuesto de 200. Los dos transformadores solo disponen de cuatro evaluaciones, frente a las diez de V0 y V1.

Tabla 13. Cobertura de los entrenamientos y mejor puntuación en el conjunto de selección. Los valores son estimaciones optimistas, no resultados finales.

Variante	Estrategia	Épocas	Eval.	Mejor puntuación	IC 95 %	Época
V0	Desde cero	500	10	$0,864 \pm 0,040$	[0,787; 0,942]	350
V1	Congelado	500	10	$0,668 \pm 0,053$	[0,564; 0,771]	150
V2	Ajuste fino	266	6	$0,648 \pm 0,057$	[0,536; 0,759]	150
V3	Congelado	155	4	$0,622 \pm 0,053$	[0,518; 0,727]	100
V4	Congelado	200	4	$0,535 \pm 0,059$	[0,419; 0,651]	100

La puntuación se acompaña de su error estándar sobre los 50 episodios evaluados. En V2 se excluye la época de índice 266, interrumpida tras cinco lotes y sin validación. V3 completó 155 épocas, es decir hasta la de índice 154, de un presupuesto de 300.

Fuente: elaboración propia.

El valor reportado es el máximo de una serie de evaluaciones y, por tanto, un estimador optimista. La media de las tres últimas ofrece una lectura menos sesgada: 0,862 en V0, 0,547 en V1, 0,585 en V2, 0,572 en V3 y 0,483 en V4. V0 permanece estable, mientras que las cuatro variantes preentrenadas retroceden después de su máximo.

Ese sesgo admite una cuantificación y no solo una advertencia. Además, el número de evaluaciones no fue el mismo, de modo que podría favorecer a las variantes con más oportunidades. La Tabla 14 reúne tres lecturas que no dependen del máximo, calculadas sobre los registros ya existentes y sin simulación adicional.

Las tres lecturas conservan la ordenación. A época común, V0 obtiene 0,828 frente a 0,668, 0,648, 0,579 y 0,435, de modo que su ventaja crece en lugar de reducirse. Con las oportunidades igualadas a cuatro evaluaciones, V0 conserva 0,837 y las cuatro variantes restantes no cambian de punto de control, ya que sus máximos aparecen dentro de las cuatro primeras evaluaciones.

El optimismo estimado varía entre 0,007 y 0,045, un orden de magnitud por debajo de las diferencias entre V0 y las demás variantes. Conviene precisar que no crece de forma ordenada con K : V2 dispuso de seis evaluaciones y presenta el menor optimismo, mientras que V1, con diez, presenta el mayor. Lo que determina la magnitud del sesgo no es solo el

Tabla 14. Lecturas del conjunto de selección que no dependen del máximo.

Variante	K	Máximo (época)	Época 150	$K = 4$ (época)	Optimismo
V0	10	0,864 (350)	0,828	0,837 (100)	0,040
V1	10	0,668 (150)	0,668	0,668 (150)	0,045
V2	6	0,648 (150)	0,648	0,648 (150)	0,007
V3	4	0,622 (100)	0,579	0,622 (100)	0,030
V4	4	0,535 (100)	0,435	0,535 (100)	0,021

K es el número de evaluaciones disponibles. La columna de la época 150 aplica una regla que no consulta las puntuaciones, puesto que 150 es la última época evaluada por las cinco variantes. La columna $K = 4$ repite la búsqueda del máximo con las cuatro primeras evaluaciones de cada una. El optimismo se estima seleccionando sobre 25 episodios y midiendo sobre los 25 restantes, promediado en 1.000 particiones.

Fuente: elaboración propia.

número de oportunidades, sino la forma de la trayectoria, puesto que un máximo aislado se corrige más que una meseta. La cifra es además conservadora: seleccionar sobre la mitad de los episodios introduce más ruido que hacerlo sobre los 50.

En conjunto, estas lecturas acotan el sesgo, pero no lo sustituyen por una medida independiente. Esa medida es la del Apartado 4.3.

V2 se reanudó desde un punto de control intermedio. El registro conserva tanto cinco épocas de la rama abandonada como sus equivalentes tras la reanudación, con pasos globales repetidos. Para construir las curvas se mantuvo la primera sesión hasta la época 69 y la rama reanudada desde la 70. Este criterio evita promediar estados del optimizador que no pertenecen a una misma trayectoria de entrenamiento.

4.2 Evolución del rendimiento

La Figura 3 representa exclusivamente las evaluaciones oficiales sobre las 50 condiciones reservadas. Las barras verticales indican el error estándar de la media sobre esos episodios. Los marcadores se sitúan cada 50 épocas y las estrellas identifican el punto de control seleccionado para cada variante.

V0 alcanza 0,8370 en la época 100 y permanece cerca de 0,86 durante la mayor parte del entrenamiento posterior. Su máximo, 0,8645, aparece en la época 350; la última evaluación, en la 450, conserva 0,8588. La pequeña diferencia entre ambos valores indica que la selección del punto de control no depende de un pico aislado. Ese máximo equivale al 96,9 % de la puntuación media que el operador humano alcanza en las demostraciones, 0,892 según el Apartado 3.2. La comparación es orientativa, ya que las condiciones iniciales no coinciden, pero acota el margen de mejora que el conjunto de datos deja disponible.

V1 mejora hasta 0,6676 en la época 150. Después oscila brevemente alrededor de 0,65 y desciende hasta 0,5536 en la última evaluación. Esta reducción equivale a 0,1140 puntos, un 17,1 % respecto a su máximo. En consecuencia, las 350 épocas restantes no producen un punto de control mejor.

V2 presenta una convergencia más lenta. Su puntuación permanece alrededor de 0,11 hasta la época 50, asciende a 0,4253 en la 100 y alcanza 0,6477 en la 150. Las dos evaluaciones posteriores caen a 0,5487 y 0,5590. Este comportamiento motiva la detención anticipada, puesto que prolongar la ejecución no había recuperado el máximo tras 100 épocas.

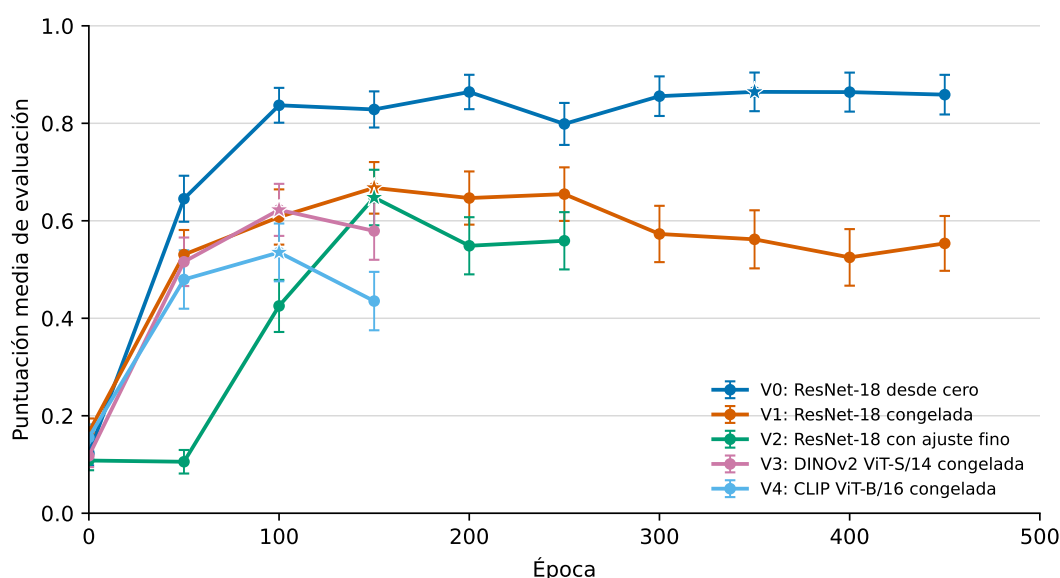


Figura 3. Evolución de la puntuación media de evaluación de las cinco variantes.

Fuente: elaboración propia.

V3 progresa de 0,1177 a 0,6224 entre las épocas 0 y 100. La evaluación de la época 150 desciende a 0,5792, y la ejecución se detiene cuatro épocas después. V4 alcanza 0,5351 en la época 100 y cae a 0,4353 en la 150, antes de completar las 200 épocas asignadas.

La comparación de las trayectorias sitúa V0 por encima de las demás variantes en todas las evaluaciones a partir de la época 50. Esta figura describe el comportamiento del entrenamiento y no constituye el resultado del estudio, puesto que las mismas puntuaciones decidieron qué punto de control se conserva. Los contrastes entre variantes se calculan sobre el bloque de prueba y se recogen en el Apartado 4.3.

4.3 Prueba final sobre semillas disjuntas

Los cinco puntos de control congelados se evaluaron una sola vez sobre las 200 condiciones iniciales del bloque 200.000 a 200.199, ajenas tanto a las demostraciones como a la selección, bajo el protocolo del Apartado 3.8. La Tabla 15 recoge el resultado, que es el resultado principal de este trabajo.

La ordenación de las cinco variantes coincide con la observada en el conjunto de selección, de modo que ninguna conclusión cambia de sentido. El error estándar se reduce de 0,040–0,059 a 0,018–0,029, consecuencia de cuadruplicar el número de condiciones.

El sesgo optimista se materializa de forma desigual. V0 obtiene sobre el bloque disjunto 0,872, ligeramente por encima de su puntuación de selección, mientras que las cuatro variantes preentrenadas pierden entre 0,019 y 0,062 puntos. En consecuencia, la ventaja de V0 no procedía de haber dispuesto de más oportunidades de selección: al medirla sin sesgo, esa ventaja aumenta.

Conviene contrastar este dato con la estimación de la Tabla 14. El optimismo realizado no reproduce el estimado por partición mitad-mitad variante a variante, y V2 ilustra la

Tabla 15. Puntuación sobre el bloque de prueba disjunto, 200 condiciones por variante.

Variante	Estrategia	Puntuación	IC 95 % BCa	Éxito	IC 95 % Wilson	Selección
V0	Desde cero	0,872 ± 0,018	[0,833; 0,903]	101/200	[0,436; 0,574]	0,864
V1	Congelado	0,649 ± 0,026	[0,596; 0,699]	37/200	[0,137; 0,245]	0,668
V2	Ajuste fino	0,586 ± 0,029	[0,528; 0,640]	32/200	[0,116; 0,217]	0,648
V3	Congelado	0,578 ± 0,028	[0,522; 0,631]	32/200	[0,116; 0,217]	0,622
V4	Congelado	0,490 ± 0,029	[0,434; 0,547]	16/200	[0,050; 0,126]	0,535

La puntuación se acompaña de su error estándar sobre las 200 condiciones. Su intervalo procede de un *bootstrap* BCa de 10.000 remuestreos con semilla 42, el mismo método que emplean los intervalos de las diferencias de la Tabla 16. La columna de éxito cuenta las condiciones que alcanzan el umbral de cobertura, y su intervalo es el de Wilson para una proporción binomial. La tasa de éxito es un resultado secundario y descriptivo: no entra en la familia de contrastes corregida. La última columna reproduce la puntuación de selección de la Tabla 13 para facilitar la comparación.

Fuente: elaboración propia.

discrepancia: el método le atribuía 0,007 y pierde 0,062. La estimación acota el orden de magnitud del fenómeno, pero no predice su valor en un caso concreto, de manera que no sustituye a la medida independiente.

La Tabla 16 recoge los diez contrastes por pares sobre ese mismo bloque.

Tabla 16. Diferencias pareadas sobre el bloque de prueba disjunto.

Comparación	Diferencia media	IC 95 % BCa	$p_{\text{Holm perm.}}$	$p_{\text{Holm Wilcoxon}}$
V0 frente a V1	0,223	[0,170; 0,279]	1×10^{-3}	$1,8 \times 10^{-12}$
V0 frente a V2	0,286	[0,228; 0,346]	1×10^{-3}	$1,7 \times 10^{-17}$
V0 frente a V3	0,294	[0,234; 0,353]	1×10^{-3}	$2,2 \times 10^{-16}$
V0 frente a V4	0,382	[0,318; 0,444]	1×10^{-3}	$2,6 \times 10^{-20}$
V1 frente a V2	0,063	[0,001; 0,124]	0,093	0,132
V1 frente a V3	0,070	[0,006; 0,132]	0,093	0,052
V1 frente a V4	0,159	[0,092; 0,224]	1×10^{-3}	$3,6 \times 10^{-5}$
V2 frente a V3	0,007	[-0,052; 0,066]	0,80	0,68
V2 frente a V4	0,095	[0,027; 0,161]	0,040	0,052
V3 frente a V4	0,088	[0,021; 0,154]	0,040	0,051

Diferencias sobre las 200 condiciones comunes, con el ruido de difusión sincronizado entre variantes. Los intervalos proceden de un *bootstrap* BCa de 10.000 remuestreos y no incorporan el ajuste de multiplicidad. La penúltima columna recoge la prueba de permutación por inversión de signo sobre la diferencia media, con 10.000 permutaciones; su valor p menor alcanzable es 1×10^{-4} , de modo que 1×10^{-3} indica el mínimo tras la corrección y no un valor estimado. La última columna recoge la prueba de Wilcoxon preregistrada, con aproximación normal y corrección de continuidad. Ambas familias de diez valores p se ajustan por separado mediante el procedimiento de Holm.

Fuente: elaboración propia.

La tabla recoge dos contrastes y conviene explicar por qué. El preregistro fijó la prueba de los rangos con signo de Wilcoxon, que opera sobre los rangos de las diferencias pareadas y no sobre su media. Como el estimando primario declarado es una diferencia de medias, esa prueba no apunta exactamente a la cantidad que se reporta. La prueba de permutación por inversión de signo sí lo hace: su estadístico es la propia media de las diferencias, y su hipótesis nula es la simetría de esas diferencias en torno a cero. Se añade por ese motivo y se declara de forma expresa como **análisis post hoc**, no preregistrado; el contraste preregistrado se conserva junto a ella y ninguno sustituye al otro.

Las cuatro diferencias entre V0 y las variantes preentrenadas son muy superiores al umbral familiar con ambos contrastes. Wilcoxon las sitúa entre $1,8 \times 10^{-12}$ y $2,6 \times 10^{-20}$; la permutación alcanza su valor p mínimo, 1×10^{-4} con 10.000 permutaciones, que tras la corrección de Holm queda en 1×10^{-3} . Los intervalos BCa, entre 0,170 y 0,444, excluyen el cero con holgura.

La conclusión principal del trabajo se sostiene, por tanto, sobre una medida que no participó en la elección de los puntos de control y no depende de qué contraste se elija.

Entre las cuatro variantes preentrenadas el cuadro es distinto y merece cautela. V4 queda por debajo de las otras tres con ambos contrastes o con uno solo: V1 frente a V4 supera el umbral con holgura en los dos ($p_{\text{Holm}} = 3,6 \times 10^{-5}$ y 1×10^{-3}), mientras que V2 frente a V4 y V3 frente a V4 lo superan con la prueba de permutación ($p_{\text{Holm}} = 0,040$) y se quedan justo por encima con Wilcoxon (0,052 y 0,051). Las tres comparaciones restantes, V1 frente a V2, V1 frente a V3 y V2 frente a V3, no alcanzan el umbral con ninguno de los dos.

De ahí se sigue una lectura prudente. Que dos contrastes discrepen a uno y otro lado de 0,05 en los mismos pares significa que esas diferencias no son robustas a la elección del procedimiento, y no que estén establecidas. La afirmación que este trabajo sostiene es la más débil de las dos posibles: V4 tiende a situarse por debajo del resto, mientras que el orden entre V1, V2 y V3 no queda resuelto. Tampoco se afirma lo contrario: la ausencia de significación no demuestra equivalencia, puesto que el diseño no fijó ningún margen ni incluyó una prueba dirigida a ello.

Merece atención aparte el contraste entre V1 y V2, congelación frente a ajuste fino. Sobre el conjunto de selección la diferencia era de 0,020 con $p = 0,82$; sobre el bloque de prueba asciende a 0,063, con p_{Holm} de 0,132 y 0,093. La evidencia apunta en contra del ajuste fino con más claridad que antes, sin alcanzar significación. Y hay una segunda razón, ajena a la estadística, para no leer esa diferencia como el efecto de la estrategia de adaptación: según se documenta en el Apartado 3.4, V1 y V2 no parten del mismo archivo de pesos preentrenados, de modo que el contraste combina la estrategia con el origen de la inicialización.

Estos resultados no cuantifican la variabilidad entre semillas de entrenamiento, cuestión que se retoma en el Apartado 4.8.

4.4 Ajuste y generalización

La puntuación de la tarea debe interpretarse junto con las pérdidas del modelo. La Figura 4 muestra los valores agregados al final de cada época completa. La escala logarítmica permite observar simultáneamente la fase inicial y los cambios de menor magnitud al final del entrenamiento.

Las cinco variantes reducen de forma sostenida la pérdida de entrenamiento. Sin embargo, la pérdida de validación vuelve a crecer después de las primeras épocas. El patrón es especialmente claro en V1: entre las épocas 150 y 450, la pérdida de entrenamiento baja de 0,0145 a 0,00166, mientras que la de validación sube de 0,0869 a 0,2247 y la puntuación de evaluación disminuye.

V2 muestra la misma divergencia. Entre las épocas 150 y 250, su pérdida de entrenamiento pasa de 0,0115 a 0,00587, la de validación aumenta de 0,277 a 0,380 y la puntuación pierde 0,089 puntos. Ambos casos son compatibles con sobreajuste. V0 también incrementa su pérdida de validación, pero su puntuación permanece estable; por tanto, la pérdida de predicción del ruido no actúa como sustituto directo del rendimiento en bucle cerrado.

V3 y V4 reproducen esa divergencia. Entre las épocas 100 y 150, la pérdida de entrenamiento de V3 baja de 0,0218 a 0,0155, mientras la de validación sube de 0,0681 a 0,0865. En V4, las mismas magnitudes pasan de 0,0143 a 0,00741 y de 0,110 a 0,171. Ambas puntuaciones disminuyen a la vez.

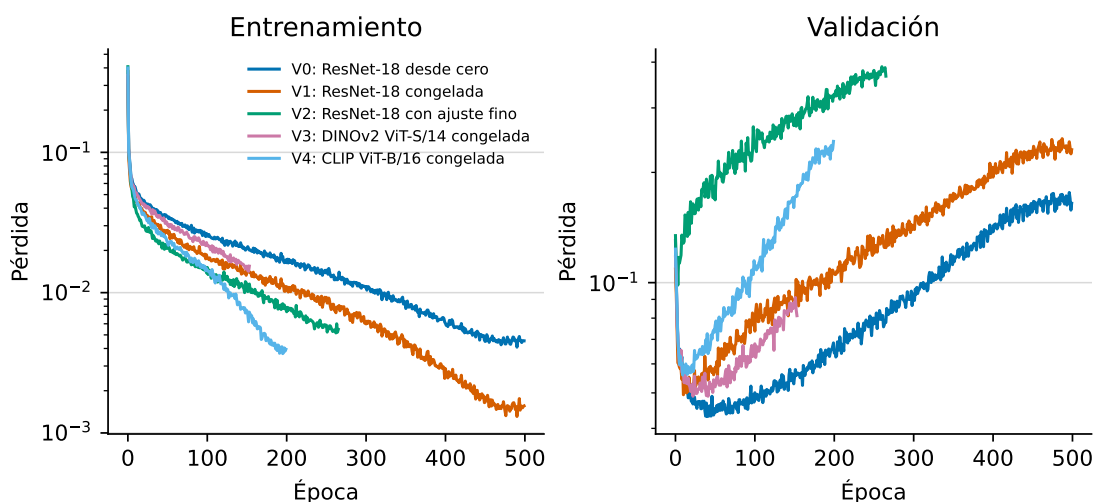


Figura 4. Evolución de las pérdidas de entrenamiento y validación.

Fuente: elaboración propia.

El error cuadrático medio de la acción refuerza esa distinción. En los puntos de control seleccionados vale 116,17 para V0, 147,31 para V1, 42,86 para V2, 314,78 para V3 y 94,93 para V4. V2 obtiene el menor error, pero no la mejor puntuación. La precisión sobre acciones registradas y el control en bucle cerrado miden, por tanto, propiedades diferentes.

4.5 Coste computacional

Los tiempos recogidos en la Tabla 12 muestran diferencias amplias. El bucle de V0 requiere 1,6 min por época, frente a 5,3 min en V1, 14,7 min en V2, 2,3 min en V3 y 4,0 min en V4. Estos valores multiplican el coste de la línea base por factores entre 1,5 y 9,5.

V2 añade la retropropagación a través del codificador y presenta el mayor coste por época. V3 y V4 duplican el número de lotes al reducir su tamaño a 32 muestras, si bien la acumulación de gradiente mantiene las actualizaciones por época en las 168 comunes a las cinco variantes. V3 muestra, sin embargo, que operar a 224 píxeles no determina por sí solo el tiempo: su codificador congelado queda más cerca de V0 que de V1.

Conviene no confundir ese coste unitario con el tiempo total. V2 acumula 84,9 h frente a las 96,5 h de V1 pese a ser la variante más cara por época, porque se detuvo antes. V3 solo suma 10,9 h por la misma razón. La comparación válida entre variantes es el tiempo por época; el total de 237,1 h describe el experimento ejecutado con presupuestos distintos.

Latencia de inferencia

El tiempo de entrenamiento no describe el coste de usar la política una vez ajustada. La Tabla 17 recoge la latencia de una llamada completa, medida sobre los puntos de control seleccionados y bajo el protocolo del Apartado 3.7.

La llamada completa apenas distingue las variantes. Con lote unitario oscila entre 1.723,8 ms en V2 y 1.749,1 ms en V4, un margen del 1,5 % que queda dentro de la dispersión de las propias medidas. La causa es estructural: cada llamada ejecuta el codificador una vez y la

Tabla 17. Latencia de inferencia de las cinco variantes, en milisegundos.

Variante	Píxeles	Lote de 1 muestra			Lote de 8 muestras		
		Llamada	Codificador	Por acción	Llamada	Codificador	Por acción
V0	76	1.743,9	3,0	218,0	2.080,8	3,1	260,1
V1	224	1.727,1	2,7	215,9	2.094,6	10,5	261,8
V2	224	1.723,8	2,8	215,5	2.096,2	10,6	262,0
V3	224	1.731,9	8,4	216,5	2.168,3	44,1	271,0
V4	224	1.749,1	16,1	218,6	2.253,9	131,8	281,7

Mediana de 50 repeticiones cronometradas, precedidas de 20 descartadas y alternadas entre variantes por rondas. El recorrido intercuartílico no supera los 82,6 ms en la columna de la llamada ni los 11,6 ms en la del codificador. Condiciones: 100 pasos de difusión, 2 observaciones de entrada, 8 acciones de salida, precisión float32, entorno de inferencia sobre Windows con PyTorch 2.6.0 y CUDA 12.4. La columna de píxeles indica la resolución que recibe el codificador.

Fuente: elaboración propia.

red generativa cien, de modo que el tiempo lo domina un componente idéntico en las cinco variantes. Una tabla que solo recogiera esa columna sugeriría que el codificador no influye en el coste de inferencia, conclusión que las dos columnas siguientes desmienten.

El codificador aislado sí ordena las variantes. Con lote unitario va de 2,7 ms en V1 a 16,1 ms en V4, un factor de 5,9. Con lote de ocho muestras la separación se amplía hasta un factor de 43, entre 3,1 ms en V0 y 131,8 ms en V4. El contraste entre ambos lotes también revela el efecto de la resolución: con una sola muestra, V0, V1 y V2 se separan menos de un milisegundo pese a operar a 76 y 224 píxeles, porque a ese tamaño domina el coste de lanzar los núcleos y no el de la aritmética; con ocho muestras, V0 mantiene sus 3,1 ms mientras V1 y V2 se van a 10,5 ms. En conjunto, el sobre coste del codificador existe y sigue el orden esperado, pero se diluye en el muestreo iterativo.

La tercera columna traduce el resultado a términos de control. Cada llamada emite ocho acciones, de modo que el coste por acción se sitúa entre 215,5 ms y 218,6 ms, equivalente a unas 4,6 acciones por segundo. El simulador consume esas ocho acciones a 10 Hz, esto es, en 0,8 s, frente a los 1,7 s que cuesta generarlas. Ninguna de las cinco variantes alcanza el control en tiempo real sobre este equipo con 100 pasos de difusión, y la diferencia entre codificadores no cambia esa conclusión: el margen está en el número de pasos del muestreo, sobre el que actúan los planificadores acelerados y la destilación revisados en el Apartado 2.2.

Memoria gráfica

El último indicador anunciado en el Apartado 3.7 es el pico de memoria gráfica reservada. La Tabla 18 lo recoge junto al pico en inferencia, que describe lo que exigiría un despliegue.

V2 es el caso extremo y el más informativo. Su paso de optimización reserva 10,293 gibibytes frente a los 8 de la tarjeta, un 128,7 % de la capacidad disponible. La ejecución no aborta porque el controlador respalda el exceso con memoria del sistema, pero cada acceso que cae fuera de la tarjeta se paga en tiempo. Ahí está la explicación de sus 14,7 min por época, casi el triple que V1, con la que comparte arquitectura, pesos iniciales, resolución y tamaño de lote. Lo único que las separa es la retropropagación a través del codificador.

Congelar el codificador, en cambio, no reduce el pico por sí solo. V1 reserva 6,896 gibibytes,

Tabla 18. Pico de memoria gráfica reservada, en gibibytes.

Variante	Codificador	Entrenamiento			Inferencia	
		Lote	Reservado	Ocupación	Lote 1	Lote 8
V0	Desde cero	64	6,447	80,6 %	1,348	1,307
V1	Congelado	64	6,896	86,2 %	1,355	1,395
V2	Ajuste fino	64	10,293	128,7 %	1,355	1,375
V3	Congelado	32	6,252	78,2 %	1,385	1,461
V4	Congelado	32	7,223	90,3 %	1,646	1,807

Memoria reservada por el asignador de PyTorch sobre una tarjeta de 8 gibibytes; la ocupación es su cociente con esa capacidad. Medida sobre 12 pasos de optimización completos, con el lote propio de cada variante, en el entorno de WSL empleado para entrenar, con PyTorch 1.12.1 y CUDA 11.6. Cada combinación de variante y modo se midió en un proceso independiente. V3 y V4 acumulan el gradiente durante dos lotes, de modo que su lote efectivo también es de 64 muestras.

Fuente: elaboración propia.

por encima de los 6,447 de V0, porque opera a 224 píxeles en lugar de 76. Lo que la congelación ahorra son el gradiente y el estado del optimizador del codificador, once millones de parámetros frente a los 277 millones del resto de la política; las activaciones del paso hacia delante se calculan igual. La memoria la determinan, por tanto, la resolución de entrada y el tamaño de lote antes que la estrategia de entrenamiento, con la salvedad del ajuste fino, que añade el grafo de retropropagación completo.

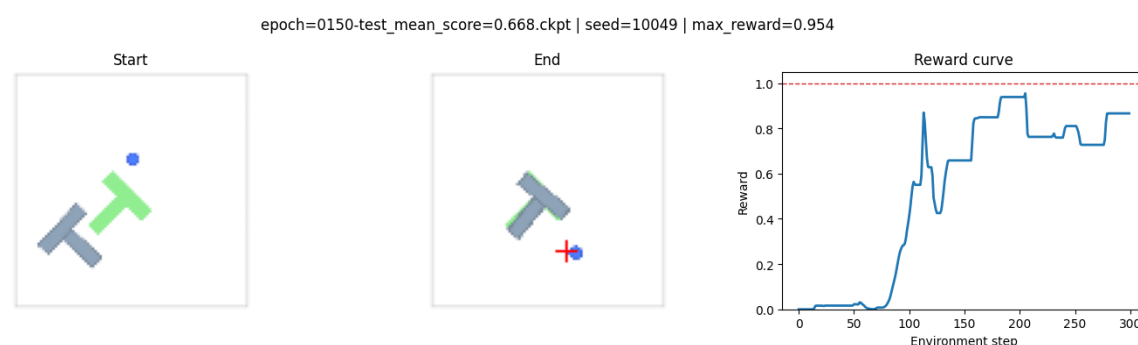
Los dos transformadores se midieron con su lote de 32 muestras. V3 reserva 6,252 gibibytes y V4, 7,223, el 90,3 % de la tarjeta. Esa cifra de V4 respalda la reducción de lote descrita en el Apartado 3.5; la de V3 deja bastante más margen, de modo que en su caso la reducción se explica mejor como cautela y como forma de mantener idénticas las condiciones de los dos transformadores. En inferencia el panorama se invierte: las cinco variantes ocupan entre 1,307 y 1,807 gibibytes, esto es, entre el 16 y el 23 % de la tarjeta. El coste de memoria de este experimento es un problema de entrenamiento y no de despliegue.

4.6 Comprobación cualitativa

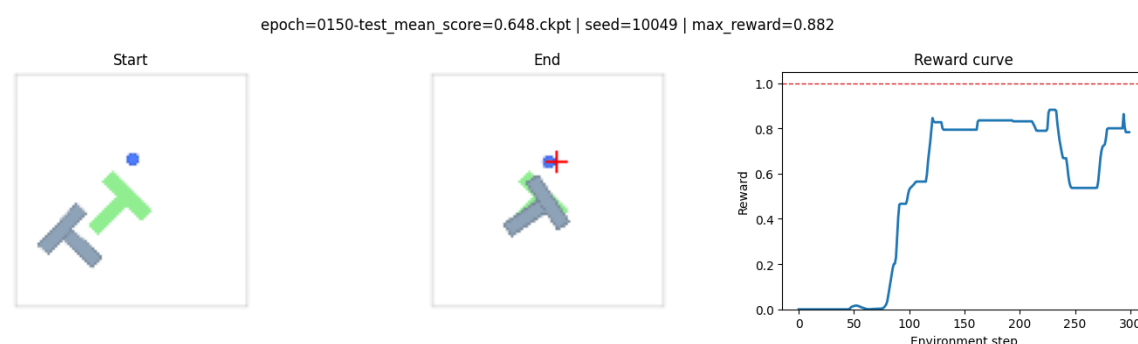
Los cuadernos de inferencia permiten inspeccionar trayectorias individuales de los mejores puntos de control. La Figura 5 compara V1 y V2 desde la misma condición inicial, generada con la semilla 10.049. Cada panel muestra el estado inicial, el estado final y la puntuación instantánea durante el episodio.

V1 alcanza una puntuación máxima de 0,9544, frente a 0,8818 en V2. Ninguna de las dos trayectorias llega al umbral unitario de éxito, aunque V1 consigue una superposición mayor. V0 sí alcanza 1,0 en su cuaderno, pero parte de la semilla 10.000; por ese motivo, su trayectoria no se incorpora a la comparación visual.

Estas demostraciones no sustituyen la evaluación cuantitativa. Sus semillas quedan fuera del intervalo oficial 100.000–100.049 y cada imagen representa un solo episodio. Además, la puntuación mostrada es el máximo temporal, no el valor del estado final. Su función es ilustrar el comportamiento que origina la métrica, no estimar el rendimiento medio.



(a) V1, ResNet-18 preentrenada y congelada.



(b) V2, ResNet-18 preentrenada con ajuste fino.

Figura 5. Inferencia de V1 y V2 sobre una condición inicial compartida.

Fuente: elaboración propia.

4.7 Contraste de las expectativas previas

El contraste se realiza sobre el bloque de prueba disjunto, no sobre el conjunto de selección. Las cifras que siguen proceden, por tanto, de la Tabla 15 y de la Tabla 16. Conviene recordar el nivel al que se formulan: las tres expectativas del Apartado 3.1 se enuncian sobre las configuraciones completas efectivamente entrenadas, y su confrontación con los datos describe lo ocurrido con esos cinco artefactos, no lo que cabría esperar al repetir el entrenamiento.

La expectativa E1 anticipaba que las configuraciones con codificador preentrenado superarían a la entrenada desde cero. Los resultados apuntan en sentido contrario: V0 supera entre 0,223 y 0,382 puntos a las cuatro, y las cuatro diferencias conservan significación tras el ajuste de Holm por amplio margen y con ambos contrastes. Es el resultado negativo más claro del trabajo.

La expectativa E2 anticipaba una ventaja del ajuste fino sobre la congelación entre las dos configuraciones que comparten arquitectura. Tampoco se cumple, ya que V1 obtiene 0,649 y V2 0,586. La diferencia de 0,063 favorece a la congelación y su intervalo, $[0,001; 0,124]$, excluye el cero por poco, pero los valores ajustados, 0,132 y 0,093, no alcanzan el umbral familiar. Además, y como se razona en el Apartado 4.3, ese contraste no aísla la estrategia de adaptación, porque las dos configuraciones no parten del mismo archivo de pesos. La lectura sostenible es que el ajuste fino no mejoró a la congelación en este experimento, sin que ello establezca nada sobre la estrategia en general.

La expectativa E3 anticipaba un coste de inferencia superior en los transformadores y dejaba abierto el orden interno entre las cuatro configuraciones preentrenadas. Sobre el rendimiento, V3 y V4 quedan por debajo de V0 y el orden entre las preentrenadas solo se resuelve en parte: V4 se sitúa por debajo de las otras tres, mientras que V1, V2 y V3 no se distinguen entre sí con ninguno de los dos contrastes. Los datos no muestran, por tanto, una ventaja de rendimiento propia de la arquitectura ViT.

Su segunda parte, la relativa al coste, se cumple en el codificador y no en la política. La Tabla 17 sitúa el codificador de V3 y V4 entre tres y seis veces por encima del de las variantes convolucionales, y la distancia crece con el tamaño de lote. La llamada completa, en cambio, apenas varía un 1,5 % entre las cinco, porque los 100 pasos de difusión diluyen esa diferencia. El mayor coste de inferencia de los transformadores es real y medible, pero no se traduce en un coste apreciable de la política en la configuración evaluada.

4.8 Limitaciones

La primera limitación es la desigualdad del presupuesto de épocas. V0 y V1 acumulan diez evaluaciones, V2 seis, y V3 y V4 solo cuatro. V3 se interrumpió tras 155 épocas de las 300 previstas, de modo que no puede descartarse una recuperación posterior. La comparación corresponde a los mejores puntos observados dentro de esos presupuestos, no a curvas con igual horizonte.

Se ensayó una prolongación de V3 y V4 destinada a igualar ese horizonte, que finalmente se descartó. La reanudación resultó defectuosa, puesto que el planificador de la tasa de aprendizaje no retomaba la posición que le correspondía y las épocas añadidas quedaban entrenadas bajo un régimen distinto del previsto. Los resultados de este capítulo proceden, en consecuencia, del protocolo original, y la desigualdad de horizontes permanece.

La segunda limitación es el uso de una sola semilla de entrenamiento, y es la más restrictiva de todas. Los intervalos y los contrastes de la Tabla 16 cuantifican la dispersión debida a las condiciones iniciales del simulador, ya que las 200 condiciones de prueba se resuelven con el mismo modelo. No cuantifican, en cambio, la variabilidad entre ejecuciones de entrenamiento, puesto que cada variante se ha ajustado una sola vez. Por tanto, las cifras describen el rendimiento de cinco artefactos concretos y no el rendimiento esperado de volver a entrenar cada estrategia. Una réplica con otra semilla podría desplazar las curvas, y esa incertidumbre queda fuera de lo reportado.

La tercera afecta al conjunto de puntos de control candidatos. El bloque de prueba del Apartado 4.3 es disjunto del de selección y proporciona una medida independiente de cada punto de control congelado. Sin embargo, durante el entrenamiento solo se guardaron los tres mejores de cada variante según la puntuación de selección, de modo que el candidato evaluado procede de un conjunto ya filtrado por la métrica contaminada. El procedimiento acota el sesgo de selección y elimina el sesgo del máximo, pero no garantiza que el punto de control congelado sea el mejor que la variante podría haber ofrecido.

La cuarta atañe a la estocasticidad de la inferencia. Una *Diffusion Policy* parte de ruido y añade ruido en cada uno de sus 100 pasos, de modo que su resultado depende tanto de la condición inicial como de la realización concreta de ese ruido. El protocolo resuelve cada condición con una única trayectoria de difusión, comun a las cinco variantes. Esa sincronización es deliberada y beneficiosa para el contraste pareado, porque elimina de la

diferencia una fuente de variación ajena a lo que se compara, pero tiene una contrapartida que debe declararse: el estimando queda condicionado a esa realización del ruido, y los intervalos de la Tabla 15 no cubren la variabilidad que aparecería al repetir la inferencia con otras semillas. Las comparaciones de diferencia pequeña, en particular las que enfrentan a V1, V2 y V3, son las más sensibles a esta limitación.

La quinta afecta al diagnóstico de sobreajuste. La pérdida de validación se estima sobre cuatro episodios, es decir 404 ventanas que no son independientes entre sí, puesto que las de un mismo episodio comparten trayectoria. Ese tamaño basta para vigilar la marcha del ajuste, pero no para sostener una afirmación cuantitativa sobre generalización. En consecuencia, las curvas de validación de la Figura 4 se emplean aquí como diagnóstico exploratorio, y el argumento sobre el sobreajuste se apoya de forma principal en las curvas de evaluación en bucle cerrado de la Figura 3, que descansan sobre 50 condiciones independientes.

Por último, la evaluación cada 50 épocas ofrece una visión discreta de la trayectoria y puede omitir máximos intermedios. V2 añade una reanudación, y solo su tiempo acumulado requiere extrapolación parcial. Estas restricciones aconsejan interpretar las cifras como evidencia del experimento realizado, no como una ordenación universal de los codificadores.

5. CONCLUSIONES

Este capítulo interpreta los resultados del Capítulo 4 a la luz de los objetivos y las expectativas enunciados en los capítulos anteriores. En primer lugar, formula una conclusión por cada objetivo específico y precisa qué permiten afirmar los datos. A continuación delimita de forma explícita el alcance de cada afirmación. Por último, expone las líneas de continuación abiertas por el estudio.

Una advertencia recorre todo el capítulo y conviene enunciarla antes que ninguna conclusión. Cada configuración se entrenó una sola vez, según la premisa de diseño del Apartado 1.4. Todas las afirmaciones que siguen se refieren, por tanto, a cinco artefactos concretos evaluados sobre un generador de condiciones iniciales, y no al rendimiento esperado de repetir cada estrategia de entrenamiento. Donde el texto dice que una variante supera a otra debe leerse que este punto de control superó a aquel en las condiciones ensayadas.

Las cinco variantes disponen de resultados, aunque sus presupuestos de entrenamiento no son uniformes. Esta diferencia limita la comparación temporal y se incorpora a la interpretación de las puntuaciones.

5.1 Conclusiones del trabajo

Sobre la implementación de las variantes

El primer objetivo específico se cumple. Las cinco variantes del codificador visual funcionan dentro de la misma *Diffusion Policy*, entregan un vector de condicionamiento de idéntica dimensión y comparten red generativa y protocolo de evaluación. Las diferencias de presupuesto y tamaño de lote quedan documentadas, por lo que la comparación se acota al bloque visual con esas dos salvedades.

Asimismo, se han evaluado las cinco configuraciones previstas. V3 se interrumpió tras 155 épocas de un presupuesto de 300; las demás finalizaron por parada anticipada o por agotamiento del presupuesto asignado. El estudio responde así a la pregunta del Apartado 1.2, con las limitaciones causales descritas más adelante.

Sobre el rendimiento de las variantes

La conclusión principal contradice la expectativa E1: ninguno de los cuatro bloques visuales preentrenados evaluados mejora el rendimiento de la línea base en Push-T. Sobre el bloque de prueba disjunto, ajeno a la selección del punto de control, la línea base alcanza 0,872, frente a 0,649 en V1, 0,586 en V2, 0,578 en V3 y 0,490 en V4. Las diferencias respecto a V0 abarcan entre 0,223 y 0,382 puntos, sus intervalos excluyen el cero con holgura y las cuatro conservan significación tras corregir los diez contrastes mediante el procedimiento de Holm, con ambos procedimientos de contraste. Es el resultado mejor sostenido del trabajo.

Conviene subrayar de dónde procede esa evidencia. Las mismas 50 condiciones que eligieron cada punto de control habrían producido una estimación optimista, de modo que la comparación se repitió sobre 200 condiciones nunca consultadas, con el protocolo fijado por

escrito antes de ejecutarla. La ordenación no cambió y la ventaja de V0 creció, puesto que fue la única variante que no perdió puntuación al pasar al bloque disjunto.

Asimismo, la expectativa E2 tampoco se cumple. La configuración que actualiza los pesos del codificador no supera a la que los congela, puesto que V2 se queda 0,063 por debajo de V1. Esa diferencia favorece a la congelación y su intervalo excluye el cero por escaso margen, pero no alcanza significación tras la corrección familiar, con valores ajustados de 0,132 y 0,093 según el contraste. Dos cautelas acompañan a este resultado y ninguna es menor. La primera, que V1 y V2 no parten del mismo archivo de pesos preentrenados, según se documenta en el Apartado 3.9, de modo que la comparación no aísla la estrategia de adaptación. La segunda, que la incertidumbre entre semillas de entrenamiento no está cuantificada. La afirmación sostenible se limita, por tanto, a estos dos artefactos concretos: el ajuste fino no mejoró a la congelación en este experimento.

El ajuste fino sí presenta una dinámica de entrenamiento menos favorable. V2 permanece estancada alrededor de 0,11 hasta la época 50, cuando V0 y V1 ya rebasan 0,53. Además, sobreajusta antes y con mayor intensidad: en la época 250 su pérdida de validación alcanza 0,380, frente a 0,120 de V1 y 0,077 de V0. Una explicación compatible con ese patrón es que la actualización de los pesos deteriore la representación de partida sin construir otra más útil con las demostraciones disponibles, pero este trabajo **no la comprueba**: no se ha medido ninguna propiedad de las representaciones, sino únicamente pérdidas y rendimiento en bucle cerrado, y esas mismas curvas admiten otras lecturas, entre ellas el sobreajuste del conjunto de la política. Queda enunciada como hipótesis y el Apartado 5.2 indica cómo contrastarla. A ello se suma que V1 y V2 no parten del mismo archivo de pesos, según se documenta en el Apartado 3.4, de modo que la comparación entre ambas no aísla la estrategia de adaptación.

Los transformadores congelados tampoco revierten el resultado. DINOv2 ViT-S/14 supera a CLIP ViT-B/16 por 0,088 puntos, si bien la diferencia queda en el límite tras la corrección familiar ($p_{\text{Holm}} = 0,051$). De las seis comparaciones entre variantes preentrenadas, solo V1 frente a V4 la supera con holgura; tres más quedan justo por encima del umbral. El orden interno de las cuatro variantes preentrenadas sigue sin resolverse, de modo que la familia del codificador no explica por sí sola las diferencias observadas.

Una explicación plausible atiende a la distancia entre dominios. Push-T contiene imágenes sintéticas de fondo uniforme y pocas formas, mientras que ImageNet representa categorías de objetos naturales [16]. DINOv2 y CLIP amplían los datos y los objetivos de preentrenamiento, pero sus descriptores globales tampoco mejoran la política. El resultado es coherente con dos antecedentes: R3M tampoco supera al codificador entrenado de extremo a extremo en Push-T [20], [3], y el estudio con DINOv3 concluye que la transferencia depende de la tarea y de la estrategia de adaptación [7].

La atribución causal exige una precisión. V0 difiere de V2 en tres factores que varían de forma conjunta: los pesos iniciales, la resolución efectiva de entrada junto con el recorte aleatorio y la capa de normalización interna, por grupos en V0 y por lotes en las demás. Frente a V1 se añade un cuarto factor, la congelación del codificador, y frente a V3 y V4 cambia además la arquitectura. La resolución merece atención propia: V0 resuelve la tarea sobre un tensor de 76×76 píxeles, mientras que las cuatro variantes preentrenadas interpolan la misma imagen de 96×96 hasta 224×224, sin añadir información. En una tarea definida por posiciones relativas, ese cambio de escala y el aumento de datos por recorte pueden pesar tanto como la inicialización.

Las conclusiones anteriores se refieren, por tanto, al bloque visual completo (pesos, preprocesado y configuración interna de la red), y no a la inicialización aislada. Separar los efectos requiere entrenamientos adicionales, descritos en el Apartado 5.2.

Sobre el coste computacional

La línea base obtiene la mejor puntuación y el menor coste unitario. Cada época de V0 ocupa 1,6 min, frente a 5,3 min de V1, 14,7 min de V2, 2,3 min de V3 y 4,0 min de V4. Las variantes preentrenadas son, por tanto, peores y entre 1,5 y 9,5 veces más costosas por época. El tiempo total conjunto asciende a 237,1 h, aunque no sirve para ordenar variantes porque los presupuestos fueron distintos.

El sobrecoste no depende del número de parámetros entrenables. V1, V3 y V4 entrenan 277 millones, frente a los 288 millones de V0, y aun así requieren más tiempo. La resolución, la arquitectura y el tamaño de lote también intervienen. De ahí se sigue una conclusión práctica: congelar el codificador no garantiza un entrenamiento más barato.

El tercer objetivo específico se cumple con los cuatro indicadores anunciados. A los parámetros y los tiempos de entrenamiento se añaden la latencia de inferencia y el pico de memoria gráfica, ambos medidos bajo un protocolo común a las cinco variantes.

De esas dos medidas se siguen dos conclusiones que el tiempo por época no permitía alcanzar. La primera es que el codificador apenas influye en la latencia de la política. Aislado cuesta entre 2,7 ms y 16,1 ms según la variante, pero la llamada completa varía solo un 1,5 %, porque los 100 pasos de difusión dominan el tiempo. Ninguna variante alcanza el control en tiempo real sobre este equipo, y la vía para conseguirlo pasa por el muestreo y no por el bloque visual.

La segunda es que la retropropagación a través del codificador, y no el hecho de estar preentrenado, es lo que agota la memoria. V2 reserva 10,293 gibibytes sobre una tarjeta de 8 y solo se completa gracias al respaldo en memoria del sistema, lo que explica su coste por época frente a V1, con la que comparte todo salvo esa retropropagación. Las cuatro variantes restantes se mantienen entre el 78 % y el 90 % de la capacidad. Conviene acotar el alcance: V2 es la única configuración con ajuste fino del estudio, de modo que la observación descansa sobre un solo par y no sobre un diseño que varíe esa decisión en cada arquitectura. En inferencia ninguna variante supera el 23 %: la restricción de memoria de este trabajo pertenece al entrenamiento, no al despliegue.

Respuesta al objetivo general

En conjunto, la respuesta al objetivo general es clara dentro de la configuración estudiada. Con 90 demostraciones de Push-T, el artefacto obtenido al entrenar la ResNet-18 de extremo a extremo a resolución nativa domina en rendimiento y en coste por época a los cuatro artefactos preentrenados. Ni DINOv2 [18] ni CLIP [19] alteran ese resultado. La ventaja corresponde al bloque visual completo y no permite atribuir el efecto exclusivamente al preentrenamiento.

El cuarto objetivo específico pide acotar la validez de esa respuesta, y esa delimitación es parte del resultado y no un descargo. La Tabla 19 separa lo que el trabajo demuestra de lo que no, en los tres niveles pertinentes.

Tabla 19. Alcance de las conclusiones: qué queda demostrado y qué no.

Demostrado para los cinco artefactos	No demostrado para las estrategias	No demostrado fuera del simulador
El punto de control de V0 supera a los otros cuatro sobre 200 condiciones disjuntas, con diferencias de 0,223 a 0,382 y significación familiar con ambos contrastes.	Que entrenar desde cero sea, en expectativa, superior a congelar o ajustar un codificador preentrenado. Haría falta replicar cada configuración con varias semillas de entrenamiento.	Que el resultado se traslade a otra tarea, a imágenes reales o a un robot físico. No se ha ejecutado ningún experimento fuera de Push-T.
Que ese resultado no procede del conjunto que eligió los puntos de control: el bloque de prueba se consultó una sola vez, con protocolo preregistrado.	Que el preentrenamiento, la resolución, el recorte o la agregación sean por separado la causa de la diferencia. Varían de forma conjunta.	Que la ordenación se conserve con otro muestreador de difusión, otro número de pasos u otro equipo.
Que V4 se sitúa por debajo de V1, V2 y V3, y que el orden entre estas tres no queda resuelto.	Que congelar sea superior a ajustar: V1 y V2 ni siquiera parten del mismo archivo de pesos.	Que el resultado sea robusto a otra realización del ruido de inferencia: solo se evaluó una trayectoria por condición.
Que V0 es la más barata por época y que el codificador apenas influye en la latencia de la política completa.	Que el ajuste fino degrade la representación de partida. No se ha medido ninguna propiedad de las representaciones.	Que ninguna variante alcance tiempo real: se comprueba en este equipo y con estos 100 pasos, no en general.

Fuente: elaboración propia.

5.2 Trabajo futuro

Del estudio se derivan nueve líneas de continuación, ordenadas por su capacidad para afianzar o corregir las conclusiones anteriores. Las tres primeras atacan directamente los límites inferenciales declarados en el Apartado 1.4; las restantes refuerzan el protocolo.

En primer lugar, y como línea prioritaria, **replicar cada configuración con varias semillas de entrenamiento**. Es la única forma de elevar la unidad de inferencia del artefacto entrenado a la configuración, y sin ella ninguna afirmación de este trabajo puede formularse sobre estrategias. El diseño mínimo son tres semillas por variante, es decir quince ejecuciones, con el mismo protocolo y sin ninguna decisión tomada por inspección. El análisis correspondiente es jerárquico: un nivel para la variabilidad entre ejecuciones y otro para la variabilidad entre condiciones iniciales, con la media por ejecución como unidad del contraste entre variantes. A partir de las 237,1 h de la Tabla 12, el coste se estima en unas 700 h de GPU. Si el presupuesto no lo permitiera, es preferible replicar tres variantes con tres semillas que cinco variantes con una.

En segundo lugar, **uniformar el presupuesto y automatizar la parada**. El protocolo de este trabajo fijó presupuestos distintos y adoptó el criterio de parada después de observar las dos primeras ejecuciones, según se declara en el Apartado 3.5. Un diseño preespecificado debe fijar de antemano un presupuesto común expresado en *actualizaciones* y no en épocas, una regla de parada evaluada de forma automática y con idéntica paciencia en todas las variantes, y un guardado de puntos de control a intervalos fijos e independientes de la puntuación. Así, todas las variantes reciben el mismo número de oportunidades de converger y de ser seleccionadas.

En tercer lugar, aislar los factores confundidos entre la línea base y las variantes preentrenadas mediante una **ablación factorial**. Un entrenamiento adicional de ResNet-18 desde cero a 224 píxeles y sin recorte aleatorio separa el efecto de la inicialización del efecto del preprocesado. Su coste por época coincide con el de V2, del orden de 15 min, puesto que el codificador se actualiza en ambos casos y la resolución de entrada es la misma; bajo el criterio de parada, el total se sitúa en torno a las 85 h. Una segunda ejecución desde cero

a 96 píxeles y sin recorte aleatorio aislaría el efecto del aumento de datos por unas 17 h, coste equivalente al de V0. Una tercera, con normalización por lotes en lugar de por grupos, cerraría el último factor. En la misma familia de ablaciones cabe examinar la agregación espacial: las cinco variantes entregan un descriptor global, obtenido por promediado en las convolucionales y por el componente de clase en los transformadores, y un *spatial softmax*, que conserva la localización de las activaciones [26], podría resultar más adecuado en una tarea definida por posiciones relativas.

En cuarto lugar, **resolver la identidad de los pesos de partida entre V1 y V2**. Tal como se documenta en el Apartado 3.4, ambas cargan ResNet-18 preentrenadas sobre ImageNet-1k procedentes de bibliotecas y recetas distintas, de modo que su contraste no aísla la estrategia de adaptación. El experimento correcto inicializa las dos desde el mismo archivo de pesos y duplica el estado antes de congelar una copia; su coste es el de repetir V1, unas 96 h, ya que V2 podría reaprovecharse si se adopta la procedencia de `torchvision`.

En quinto lugar, ampliar el conjunto de puntos de control candidatos. La prueba del Apartado 4.3 mide sin sesgo el punto de control congelado de cada variante, pero ese candidato procede de los tres que el entrenamiento guardó según la puntuación de selección. Guardar los puntos de control a intervalos fijos, con independencia de su puntuación, y elegir después sobre un bloque de selección también disjunto cerraría ese resquicio del protocolo actual.

En sexto lugar, **replicar la inferencia con varias semillas de ruido**. La prueba final resuelve cada condición inicial con una única trayectoria de difusión, común a las cinco variantes, de modo que el estimando queda condicionado a esa realización del ruido y los intervalos no cubren la variabilidad propia del muestreo estocástico. El diseño que la estimaría repite el bloque de 200 condiciones con, por ejemplo, cinco semillas de ruido, manteniendo los números aleatorios comunes *dentro* de cada réplica para no perder el emparejamiento, y descompone la varianza en sus dos niveles. Su coste es proporcional al de la prueba ya realizada: unas 2,5 h por réplica de las cinco variantes.

En séptimo lugar, ampliar el conjunto de validación. Los cuatro episodios que fija la implementación de referencia bastan para vigilar la marcha del ajuste, pero no para sostener un diagnóstico de sobreajuste: sus ventanas son temporalmente dependientes, de modo que las 404 disponibles equivalen a bastante menos información efectiva. Un reparto con veinte episodios de validación, o una validación cruzada repetida a nivel de episodio y sin mezclar ventanas del mismo episodio entre subconjuntos, permitiría acompañar la pérdida de validación de su incertidumbre. El coste es despreciable frente al del bucle de optimización.

En octavo lugar, repetir la comparación con los 202 episodios disponibles para ajuste, y no solo con los 90 que fija la implementación de referencia. El descarte es aleatorio y representativo, según se comprueba en el Apartado 3.2, de modo que la línea mide el efecto del tamaño del conjunto y no el de su composición. Ese experimento resulta especialmente pertinente aquí, puesto que la ventaja del preentrenamiento se atribuye habitualmente a la escasez de datos. Su coste por época crecería en proporción al número de ventanas, algo más del doble del actual.

Por último, extender el protocolo a otras tareas y a un robot físico. Push-T aporta un banco de pruebas controlado, pero sus imágenes sintéticas son precisamente el escenario menos favorable para un codificador preentrenado sobre fotografías naturales. La conclusión podría invertirse en observaciones reales, extremo que este trabajo no puede resolver.

6. PRESUPUESTO

Este capítulo cuantifica el coste de ejecución del proyecto. El presupuesto recoge cuatro partidas: material fungible y suministros, amortización de los equipos, licencias de software y mano de obra. A ellas se añaden unos costes indirectos y, con todo, se obtiene el importe total.

Tres criterios de imputación gobiernan las cifras que siguen. En primer lugar, el periodo imputado abarca siete meses, de marzo a septiembre de 2026, que comprenden desde la revisión bibliográfica hasta la entrega de la memoria. En segundo lugar, los equipos se amortizan de forma lineal y se imputa la fracción correspondiente a esos siete meses, con independencia de su uso efectivo dentro de cada jornada. En tercer lugar, los importes se expresan sin impuestos indirectos, puesto que el trabajo se desarrolla en el ámbito académico y no da lugar a facturación.

6.1 Material fungible y suministros

El proyecto es de naturaleza computacional, de modo que apenas consume material físico. La única partida relevante es la energía eléctrica, dominada por los entrenamientos desatendidos. Según el Apartado 3.10, estos ocuparon 237,1 h de cómputo; con un consumo aproximado de 150 W bajo carga, suman unos 36 kilovatios-hora. El resto de la dedicación añade otros 24 kilovatios-hora, con el equipo en régimen de trabajo ordinario. La Tabla 20 recoge ambos conceptos.

Tabla 20. Presupuesto de material fungible y suministros.

Concepto	Cantidad	Coste unitario (€)	Total (€)
Energía eléctrica (kWh)	60	0,15	9,00
Material de oficina e impresión	1	45,00	45,00
Total			54,00

Se adopta un precio de 0,15 €/kWh, valor de referencia para el suministro doméstico durante el periodo imputado.

Fuente: elaboración propia.

6.2 Amortización de los equipos

El proyecto se ejecuta sobre un único equipo, descrito en el Apartado 3.9. Dado que su vida útil excede la duración del trabajo, no se imputa el precio de adquisición sino la amortización correspondiente al periodo de uso, según la Ecuación (7).

$$A = C \cdot \frac{t}{V} \quad (7)$$

En la Ecuación (7), A es el importe imputado, C el precio de adquisición, t el periodo de uso y V la vida útil, ambos expresados en meses. Se aplica amortización lineal y sin valor

residual, criterio habitual para equipos de proceso de información. La Tabla 21 detalla el resultado para cada elemento.

Tabla 21. Amortización de los equipos imputada al proyecto.

Equipo	Precio (€)	Vida útil (años)	Uso (meses)	Imputación (€)
Portátil con GPU dedicada	1.800,00	4	7	262,50
Monitor externo	180,00	5	7	21,00
Unidad externa de respaldo	60,00	3	7	11,67
Total				295,17

Fuente: elaboración propia.

Conviene señalar que la unidad de procesamiento gráfico condiciona el diseño experimental, según se argumentó en el Apartado 3.9, pero apenas condiciona el presupuesto. Su amortización forma parte de los 262,50 € imputados al portátil, importe menor que el coste de dos jornadas de trabajo.

6.3 Licencias de software

Todas las herramientas empleadas son de código abierto o de uso gratuito, de modo que esta partida asciende a 0,00 €. La Tabla 10 enumera la pila de software: Python, PyTorch, torchvision, timm, diffusers, robomimic, Hydra y Zarr se distribuyen bajo licencias libres. A ellas se suman la distribución Ubuntu, el sistema de composición T_EX Live y el control de versiones Git, también libres. La licencia de Windows 11 viene incluida en el precio del portátil y queda, por tanto, recogida en el Apartado 6.2.

6.4 Mano de obra

La mano de obra distingue dos perfiles. El autor asume la implementación, la ejecución, el análisis y la redacción, con el coste horario de un ingeniero de datos junior. El director asume la definición del alcance, el seguimiento periódico y la revisión de la memoria, con el coste horario de un investigador sénior. Ambos importes corresponden al coste bruto para la entidad, incluidas las cargas sociales.

La Tabla 22 desglosa la dedicación del autor por fases. El reparto refleja el peso real de cada actividad: la preparación del entorno de cómputo resultó más costosa de lo previsto, mientras que la ejecución de los entrenamientos, aunque ocupó más de 237 horas de máquina, exigió poca intervención por tratarse de procesos desatendidos.

La dedicación del director se estima en 40 horas: unas 20 reuniones de seguimiento de una hora, repartidas a lo largo de los siete meses, y otras 20 horas de revisión de los borradores. La Tabla 23 aplica los costes horarios a ambos perfiles.

6.5 Resumen del presupuesto

La Tabla 24 agrega las cuatro partidas anteriores y añade unos costes indirectos del 15 % sobre el subtotal, porcentaje que cubre espacio de trabajo, conectividad y servicios generales no imputables a un concepto concreto.

Tabla 22. Dedicación del autor por fases del proyecto.

Fase	Horas
Revisión bibliográfica y estado del arte	70
Preparación del entorno de cómputo	55
Implementación de las variantes del codificador	90
Ejecución y supervisión de los entrenamientos	60
Análisis de resultados y comprobación cualitativa	65
Redacción de la memoria	110
Total	450

Fuente: elaboración propia.

Tabla 23. Presupuesto de mano de obra.

Perfil	Horas	Coste horario (€)	Total (€)
Ingeniero de datos junior (autor)	450	30,00	13.500,00
Investigador sénior (director)	40	60,00	2.400,00
Total	490		15.900,00

Fuente: elaboración propia.

Tabla 24. Resumen del presupuesto.

Partida	Importe (€)
Material fungible y suministros	54,00
Amortización de equipos	295,17
Licencias de software	0,00
Mano de obra	15.900,00
Subtotal	16.249,17
Costes indirectos (15 %)	2.437,38
Total	18.686,55

Fuente: elaboración propia.

El coste total del proyecto asciende, por tanto, a 18.686,55 €. La distribución entre partidas resulta muy desigual: la mano de obra representa el 85 % del total, mientras que los equipos apenas alcanzan el 1,6 %. Esa asimetría es característica de un proyecto de investigación computacional ejecutado sobre hardware propio.

De ahí se sigue una consideración económica sobre el diseño experimental. Las limitaciones descritas en el Apartado 3.10 no proceden del presupuesto, sino del tiempo de calendario: ampliar el estudio a tres semillas por variante multiplicaría por tres el cómputo sin apenas alterar el coste, ya que la amortización y la energía son partidas menores. El factor limitante es que esas horas de máquina deben transcurrir dentro del plazo del proyecto, no lo que cuestan.

REFERENCIAS

- [1] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, ser. JMLR: W&CP, vol. 15, Fort Lauderdale, FL, USA, 2011, pp. 627–635.
- [2] P. Florence, C. Lynch, A. Zeng, O. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, “Implicit behavioral cloning,” in *Proc. 5th Conf. Robot Learning (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 164, London, U.K., 2021, pp. 158–168.
- [3] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10–11, pp. 1684–1704, 2025, doi: 10.1177/02783649241273668.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, Vancouver, Canada, 2020, pp. 6840–6851.
- [5] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations,” arXiv:2403.03954, 2024, presentado posteriormente en Robotics: Science and Systems (RSS).
- [6] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, “Consistency policy: Accelerated visuomotor policies via consistency distillation,” arXiv:2405.07503, 2024.
- [7] T. Egbe, P. Wang, Z. Guo, and Z. Chen, “DINOv3-Diffusion Policy: Self-supervised large visual model for visuomotor diffusion policy learning,” arXiv:2509.17684, 2025.
- [8] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Proc. 5th Conf. Robot Learning (CoRL)*, London, U.K., 2021, arXiv:2108.03298.
- [9] Y. Song and S. Ermon, “Generative modeling by estimating gradients of the data distribution,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, Vancouver, Canada, 2019, pp. 11 895–11 907.
- [10] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021, arXiv:2011.13456.
- [11] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021, arXiv:2010.02502.
- [12] X. Ma, S. Patidar, I. Haughton, and S. James, “Hierarchical diffusion policy for kinematics-aware multi-task robotic manipulation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, arXiv:2403.03890.

- [13] X. Sun, F. Fan, Y. Chen, and D. Rakita, “Optimizing active perception for learning simultaneous viewpoint selection and manipulation with diffusion policy,” arXiv:2409.14615, 2024.
- [14] C. Tie, Y. Chen, R. Wu, B. Dong, Z. Li, C. Gao, and H. Dong, “ET-SEED: Efficient trajectory-level SE(3) equivariant diffusion policy,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025, arXiv:2411.03990.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.
- [16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Miami, FL, USA, 2009, pp. 248–255.
- [17] A. Dosovitskiy, L. Beyer, A. Kolesnikov *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021, arXiv:2010.11929.
- [18] M. Oquab, T. Darcet, T. Moutakanni *et al.*, “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024, arXiv:2304.07193.
- [19] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. Proc. Mach. Learn. Res., vol. 139, 2021, pp. 8748–8763.
- [20] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, “R3M: A universal visual representation for robot manipulation,” in *Proc. 6th Conf. Robot Learning (CoRL)*, 2022, arXiv:2203.12601.
- [21] T. Xiao, I. Radosavovic, T. Darrell, and J. Malik, “Masked visual pre-training for motor control,” arXiv:2203.06173, 2022.
- [22] S. Karamcheti, S. Nair, A. S. Chen, T. Kollar, C. Finn, D. Sadigh, and P. Liang, “Language-driven representation learning for robotics,” in *Proc. Robotics: Science and Systems (RSS)*, Daegu, Republic of Korea, 2023, doi: 10.15607/RSS.2023.XIX.032.
- [23] A. Majumdar, K. Yadav, S. Arnaud, J. Ma, C. Chen, S. Silwal, A. Jain, V.-P. Berges, T. Wu, J. Vakil, P. Abbeel, J. Malik, D. Batra, Y. Lin, O. Maksymets, A. Rajeswaran, and F. Meier, “Where are we in the search for an artificial visual cortex for embodied intelligence?” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 655–677, doi: 10.52202/075280-0031.
- [24] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. AAAI Conf. Artificial Intelligence*, New Orleans, LA, USA, 2018, arXiv:1709.07871.
- [25] Y. Wu and K. He, “Group normalization,” in *Proc. European Conf. Computer Vision (ECCV)*, Munich, Germany, 2018, pp. 3–19.
- [26] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-end training of deep visuomotor policies,” *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016.

- [27] R. Wightman, H. Touvron, and H. Jégou, “ResNet strikes back: An improved training procedure in timm,” arXiv:2110.00476, 2021.
- [28] R. Wightman, “PyTorch image models,” Repositorio de software, <https://github.com/huggingface/pytorch-image-models>, 2019.
- [29] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, New Orleans, LA, USA, 2019, arXiv:1711.05101.
- [30] M. Britez, “Repositorio de código fuente del trabajo de fin de máster: codificadores visuales en Diffusion Policy,” https://github.com/moisesbritez92/tfm_madi, 2026, revisión 8bc22e7 para el protocolo de la prueba final.
- [31] A. Paszke, S. Gross, F. Massa *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, Vancouver, Canada, 2019, pp. 8024–8035.
- [32] M. Britez, “Guía de estudio técnica, matemática y conceptual: Diffusion Policy,” Monografía autocontenida, 2026.